



Secret handshakes: Full dynamicity, deniability and lattice-based design

Zhiyuan An^{a,b}, Jing Pan^c, Yamin Wen^d, Fangguo Zhang^{a,b,*}

^a School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China

^b Guangdong Province Key Laboratory of Information Security Technology, Guangzhou 510006, China

^c Guangzhou Institute of Technology, Xidian University, Guangzhou 510555, China

^d School of Statistics and Mathematics, Guangdong University of Finance and Economics, Guangzhou 510320, China

ARTICLE INFO

Article history:

Received 30 March 2022

Received in revised form 2 September 2022

Accepted 24 October 2022

Available online 26 October 2022

Communicated by G. Yang

Keywords:

Secret handshake

Lattice cryptography

Full dynamicity

Zero-knowledge

Deniability

ABSTRACT

Secret handshake (SH), as an essential privacy-preserving primitive, entitles members of the same group to anonymously authenticate each other. Among all the existing designs, only the ones separately proposed by Zhang et al. over coding theory and An et al. over lattice are post-quantum secure. However, both schemes fall short of being practical due to their huge overhead ($\gg 100$ MB), and the code-based one is even insecure under impersonator attacks. Besides, all the known SH constructions are powerless to handle the problem of dynamically managing users. To fill the gap, in this work we first formalize the model of fully dynamic secret handshakes (FDSH), where members have the freedom of joining or leaving a group. Then, to achieve anonymous mutual authentication with ease, we introduce a newly-defined cryptographic primitive called anonymous message exchange (AME), where legal participants can exchange their messages anonymously. Further, two generic approaches of AME from non-interactive zero-knowledge arguments of knowledge (NIZKAoK) are provided, along with heuristic applications. Finally, by combining a well-designed AME and other suitable techniques, we present the first lattice-based FDSH in the random oracle model offering another attractive property: full deniability, which enables users to plausibly deny their participating in interactions. Our scheme sharply reduces the communication cost (< 10 MB) and surpasses the existing post-quantum systems in terms of security, efficiency, and functionality.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

SECRET HANDSHAKE, introduced by Balfanz et al. [5], is a fundamental anonymity primitive. In such schemes, potential members form different groups and conduct an interactive protocol to authenticate each other. The mutual handshake is successful if and only if both parties belong to the same organization. Except for the affiliations, no extra information (including their identities) about the involved members will be leaked. Therefore, secret handshakes provide all-sided privacy-preserving property for enrolled members. Nevertheless, a tracing procedure is always applied to keep users accountable for their behaviors. To date, many practical applications of secret handshakes in social networks have been explored, such as online appointment, mobile access [36] and e-healthcare [18], etc.

* Corresponding author at: School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China.
E-mail address: isszhfg@mail.sysu.edu.cn (F. Zhang).

Table 1

Comparison between known schemes and ours.

Scheme	Model	Post-quantum	Imper. attack	FD-unlink.	Full deni.	Hand. cost	Update cost	Tracing cost
[5]	Static	✗	Against	✗	✗	0.04 KB	$L \cdot 10$ b	10 b
[24]	Static	✗	Against	✗	✗	0.35 KB	—	10 b
[23]	VLR	✗	Against	✗	✗	< 1 KB	<1 KB	10 b
[47]	VLR	✗	Against	✗	✗	0.36 KB	<1 KB	10 b
[44]	Static	✗	Against	✗	*	NA	$L \cdot 10$ b	10 b
[51]	Static	✓	✗	✗	✗	171.3 MB	$L \cdot 10$ b	10 b
[3]	VLR	✓	Against	✗	✗	7.8 GB	579 KB	10 b
Ours	Fully dynamic	✓	Against	✓	✓	7.29 MB	1.54 KB	10 b (+10.93 MB)

Since most online systems offering authentication interface always operate with a high staff turnover, a desirable functionality of secret handshakes is to support membership revocation, which revokes users from their groups and publishes the withdrawal notification to the other active ones. Capturing this property in an efficient manner is somewhat challenging, as the group authority must ensure that revoked users can no longer execute a handshake, and in the meantime, the structure of the group (group public keys, users' credentials) does not undertake overmuch change. Several possible approaches were proposed to address this problem. Jarecki and Liu [22] designed a revocable SH scheme via broadcast encryption. Subsequently, in [23] they proposed a succinct framework supporting both traceability and revocation. Then Wen et al. [47] presented an improved scheme considering verifier-local revocation (VLR). However, all these attempts fail to achieve full dynamicity (the flexibility in joining and leaving a group), let alone the model syntax. As a similar privacy-preserving primitive, group signature achieving full dynamicity has been defined in [7] and efficiently constructed in [31]. Thus, it is worthwhile to exploit the analogous definition and construction for SH.

Another interesting functionality of SH we consider is (full) deniability suggested by Tian et al. [44]. Deniable SH allows legitimate users to deny their participating in finished conversations, even if the secrecy of these handshakes is compromised later. In this sense, no one can successfully prove to a judge that any user has participated in a handshake, even if she has really done that. This property has been widely explored in the context of authentication and key exchange [40,41,46], as in real-world scenarios, two entities always want to communicate with each other without producing any convincing evidence (unforgeable signatures or distinguishable session tags) of the interaction. A weak model, named strong deniability, has been proved in [44] to be inherently satisfied for some existing SH schemes [22,21,23,47]. However, full deniability, where the simulator has no access to the user's credentials or group secret key, is non-trivial to realize, and no concrete instantiation has been proposed.

For the time being, nearly all the SH schemes are designed on the hardness of factoring or discrete logarithm problem, which are vulnerable to quantum attacks. As all we know, the first post-quantum SH was proposed by Zhang et al. [51] from coding. Then An et al. [3] presented a lattice-based variant using the VLR method in [26], [3] also pointed out that due to improper use of Stern's identification system in [51], an adversary can easily impersonate an honest user to activate a handshake. Nevertheless, both schemes fall well short of being applied in practical scenarios, because of their huge overhead ($\gg 100$ MB) and rather complicated techniques (CFS signature or lattice trapdoors). Moreover, Stern-like zero-knowledge (ZK) proofs are extremely utilized to realize mutual authentication in their designs, while no explicit relationship between SH and ZK systems is explored. The unsatisfactory situation encourages us to design more efficient post-quantum SH, as well as build some generic and compact approach of mutual authentication.

OUR CONTRIBUTIONS. This work addresses the above three limitations of existing SH. Our contributions are summarized in the following.

- By adding some dynamicity ingredients and defining other desirable security properties, we adapt the static model of SH into the fully dynamic setting.
- We introduce a new cryptographic primitive: anonymous message exchange, ensuring legal participants to anonymously exchange messages. Then two generic constructions from NIZKAoK are provided, which are further instantiated to show how to achieve anonymous mutual authentication with ease.
- Finally, by using AME plus suitable accumulator and encryption methods, we present the first fully dynamic secret handshake scheme with deniability from lattice, which is free of trapdoor and greatly outweighs the available post-quantum ones in terms of functionality, security and efficiency.

Table 1 gives an integrated comparison of some existing SH and our scheme under 80-bit security level and maximum number of group members $N = 2^{10}$. For the static model, [5,44,51] applies one-time pseudonyms where we use L to denote the number of pseudonyms a user holds, L can be $O(N)$ for a large SH system, and updating the system for this type of SH means publishing a revoked user's pseudonyms into a revocation list; [24] only supports user enrollment, i.e., users cannot be revoked (so the entry "update cost" is blank). Therefore, all of these "static" constructions are unpractical. For the model of VLR [23,47,3], users need to conduct an extra revocation check whose runtime is linear on the number of revoked users, thus still being inflexible. For our fully dynamic scheme, the plain tracing algorithm only outputs the

identity of the involved user (having size 10b), but if we opt to additionally demonstrate the correctness of a tracing result (none of the previous schemes consider this functionality), a zero-knowledge proof is required (having size 10.93 MB). As for full deniability, [44] only gave a rough blueprint and no instantiation was proposed, while our scheme for the first time achieves this notion. Finally, note that the existing post-quantum designs [51,3] undertake huge communication cost and much weaker functionalities. The above results indicate that our scheme provides improved functionality and security compared with these prior works, and also drastic efficiency gain in contrast to the two post-quantum ones.

OVERVIEW OF OUR TECHNIQUES. In the following, we provide more technical details of our main contributions.

Fully Dynamic Secret Handshake. Inspired by the definition of fully dynamic group signature [7], for the first time we consider *full dynamicity* in secret handshake, where users have the freedom of joining or leaving a group. To model this property, we redefine the launching algorithm `CreateGroup` of SH: except the group key pair (mpk, msk), group authority (GA) also publishes dynamic group information info when invoking `CreateGroup` to establish a new group. info will be updated whenever there is a member change and by downloading the current one, users can autonomously update their group credentials and conduct revocation check. Compared with the previous static or VLR model, this setting supports more flexible system management but is free of impractical overhead. Besides, to better supervise user's behaviors within this dynamic state, tracing authority (TA) of FDSH also needs to generate a proof showing the correctness of a tracing result, such that no user can get off scot-free if she has successfully authenticated with others.

As for the security notions of FDSH, capabilities of the adversary are carefully considered to the greatest extent. Specifically, unlike the case in [5,12], for *detector resistance* one can corrupt all the GA and users except the challenging pair; for *impersonator resistance* and *unlinkability*, TA and the target user can be further controlled, respectively. The latter is more strict for unlinkable SH compared with backward unlinkability [37] where only users' public keys are exposed. Moreover, to capture the function of accountability, we consider another privacy requirement *traceability* which makes sure that the tracing result of a successful handshake must be a valid user w.r.t. the tracing secret key.

Achieve Full Dynamicity. Efficiently managing users' enrollment and revocation has always been a challenging task. One trivial idea is to re-initialize the whole system including group public keys and users' secret keys whenever a member change occurs, obviously it is too cumbersome for a real-world application. Another possible revocation technique is VLR [6]: users are issued a secret revocation token which will be published after they are revoked. However, VLR still loses sight of user enrollment and users have to perform a costly (in runtime) revocation check.

To cover both users' enrollment and revocation, we employ the dynamic Merkle trees [31] to issue users' credentials for our FDSH scheme. Specifically, for a group of maximum member $N = 2^\ell$, GA first builds an all-zero Merkle-tree τ . Potential user samples a random $\mathbf{x}_i \in \{0, 1\}^m$ as the user secret key usk, followed by the public key $\text{upk} := \mathbf{p}_i = \text{bin}(\mathbf{A} \cdot \mathbf{x}_i) \in \{0, 1\}^{nk}$ where $\mathbf{A}$ is public. Upon receiving upk, GA chooses a spare $i \in \{0, 1\}^\ell$ as user's identity, then updates τ by setting the related leaf as $\mathbf{p}_i$ and changing the values of adjoint path nodes. Finally GA publishes the updated info including a new root $\mathbf{u}$ and all the witnesses $\{w_j\}_j$, which are the path nodes indicating that every $\mathbf{p}_j$ can be accumulated into $\mathbf{u}$. In this way, user's group credential is $(\mathbf{p}_i, w_i)$. Finally, when revoking a malicious/expired user, GA resets the value of the related leaf as $\mathbf{0}$ and also publishes the updated root and witnesses as the new info, with which users can update their credentials by themselves.

Anonymous Message Exchange. The remaining problem is to build Handshake, the core algorithm of SH, where users can anonymously authenticate each other and negotiate a session key for further communication. This functionality requires participants to anonymously prove their valid group credentials to each other, for which anonymity-preserved signatures or ZK systems are often applied [24,23,47,51,3]. Then to further negotiate a session key, Diffie-Hellman-like key exchange (KE) is always utilized in these previous works.

To better handle this task, in this work we are going to explore some generic approach of Handshake. Namely, we break the design of Handshake down into some sub-problems. Note that during the interaction of a handshake, users intend to exchange some credential-related information, so that they can authenticate each other via checking some knowledge about the information. Clearly, it allows users having valid credentials to anonymously exchange messages with each other. Once have achieved this goal, mutual authentication can be easily realized: set the dispatching messages as the public elements in a key exchange, and then generate authentication codes (e.g., hash values) of the extracted key for verification. In this sense, AME is formalized to model the special interactive protocol for the first aim described above. More detailedly, there are three algorithms in AME: `Setup` generates group public/secret key (gpk, gsk) and users' credential cred; On inputting gpk and cred, `Send` produces a text Σ with desired message M embedded in it; `Open` recovers the hidden message from Σ by use of gpk. Here we stress that AME is not a strict one-to-one interaction, since anyone can invoke `Open` to retrieve the hidden message. On the other side, we place restrictions on who can produce a sound text: only users with legal credentials. Therefore, whether the resulting pairs of each other are identical hinges on whether they both have valid credentials. Finally, users' anonymity is captured by asking the outcoming texts to be indistinguishable from random.

Framework, Instantiation and Use of AME. Since AME implicitly requires senders to anonymously prove their valid credentials, we consider non-interactive zero-knowledge arguments of knowledge (NIZKAoK) as a feasible building block. Recall

that a general NIZKAoK consists of 3 components: a commitment cmt , a challenge $ch = \mathcal{H}(cmt, \cdot)$ binding to cmt and a response rsp used to recover part of cmt , being *zero-knowledge* makes such systems suitable for privacy-preserving protocols. However, the key problem here is how to make sure that only authorized senders can embed a message into a ZK proof, such that it can be efficiently recovered in a public way. Motivated from previous constructions [51,3] of SH, we propose a subtle strategy: instead of dispatching the whole evidence, send partial commitment $\widehat{cmt}$ which cannot be recovered by rsp , then change the original challenge into a mixture $\widehat{ch} = ch \oplus M$ and let $\Sigma = (\widehat{cmt}, \widehat{ch}, rsp)$. After these adjustments, one can first retrieve the rest of cmt using rsp , then compute the original $ch = \mathcal{H}(cmt, \cdot)$, so as to recover the hidden $M = \widehat{ch} \oplus ch$, yet the correctness of M depends on the validity of the original proof. In this way, participants can exchange their messages with implicitly confirming each other's identities. Two generic constructions of AME from Stern-like and Fiat-Shamir with abort NIZKAoK are presented to elaborate this tactic in Section 4.

Now we briefly exhibit how to build Handshake of our FDSH scheme from an instantiation of AME. When executing a handshake, users also need to encrypt ID via a binary-bit scheme with double-encryption paradigm [38] to achieve *traceability*. Therefore we aim to construct an AME scheme, where messages can be embedded into a proof demonstrating in zero-knowledge the possession of $\{cred, usk\}$, and that the ciphertexts are well-formed. This task can be done by first reducing the overall linear relations to be proved into an instance Π_{hs} of the lattice-based NIZKAoK blueprint in [50], and then transforming Π_{hs} into an AME via the aforementioned generic approach. Also as mentioned before, through setting message M as the public matrix $\mathbf{C}$ of a lattice-based KE [8], participants can leverage the well-designed AME to negotiate a session key $\mathbf{K}$ and produce authentication codes $\mathcal{H}(\mathbf{K}, \mathbf{C})$. That is exactly how algorithm Handshake works. Accordingly, the security flaw of Zhang et al.'s scheme [51] can be corrected in a similar way.

(Fully) Deniable Secret Handshake. [44] firstly brought the notion *deniability* (also known as “off-the-record”) into the field of SH, which entitles users to deny their interactions with others, i.e., no adversary $\mathcal{A}$ can retrieve some valid proof evidencing that some user of a designated group has participated in a handshake, merely from his handshake transcripts with that user. To model this property, [44] asks that there a PPT simulator, which, feeding all the public system information accessible to $\mathcal{A}$, produces a simulated and indistinguishable view from $\mathcal{A}$'s real view in a handshake. Following the definition in [44], we further extend $\mathcal{A}$'s abilities to approach the malicious contents in real-world applications as close as possible. Namely, now we allow $\mathcal{A}$ to corrupt any user or any group to obtain group credentials and secret keys except for the tracing secret key, and demand that $\mathcal{A}$ still gains no extra advantage when proving its interactions with some user. To the best of our knowledge, the aforementioned *full deniability* is the strongest deniable setting for SH, since $\mathcal{A}$ almost controls the whole system except for TA. In this aspect, our model also explains why most previous schemes fail to support *full deniability*: transcripts of handshakes in their design always leave some verifiable trails bonding to user's credentials.

We then list some brief discussions on how our lattice-based FDSH scheme can capture *full deniability*. The main idea is that we can leverage the proof simulator of the underlying NIZKAoK [50] to produce a simulated text Σ_{simu} for our lattice-based AME, note that Σ_{simu} is computationally indistinguishable from the real one, thanks to *zero-knowledge* of utilized NIZKAoK system. Then by randomly sampling the other elements used in an interactive handshake from the corresponding distributions, we can construct the well-formed view simulator required by *full deniability*.

RELATED WORKS. Following Balfanz et al.'s initial work [5], there have been numerous works on secret handshakes in the last quarter-century. Some of them employed one-time pseudonyms to ensure unlinkability [12,21,45,52,53]. This design needs GA to compute and store vast credentials, which is a heavy burden for management. Thus, a more efficient method is to apply reusable credentials for privacy protection. Xu and Yung [49] first offered a secret handshake scheme with reusable credentials having a weaker unlinkability. Ateniese et al. [4] proposed an efficient unlinkable secret handshake scheme without random oracle. Subsequently, Jarecki and Liu [23] proposed a framework for unlinkable secret handshake scheme that supports both traceability and revocation with reusable credentials. From then on, some practical secret handshake schemes meeting different authentication requirements were proposed [19,39,44,48].

ORGANIZATION. In Section 2, we recall some necessary background and techniques over lattice. Model and security definitions of fully dynamic secret handshakes are presented in Section 3. Section 4 introduces the new primitive: anonymous message exchange, together with two generic approaches and instantiations. Our main scheme is presented and analyzed in Section 5. Section 6 concludes our work.

2. Preliminaries

Notations. For a positive integer n , denote the set $\{1, \dots, n\}$ by $[n]$, and the set $\{0, 1, \dots, n\}$ by $[0, n]$. Vectors will be denoted in bold lower-case letters and matrices will be denoted in bold upper-case letters. All vectors are column vectors unless otherwise specified. For vector $\mathbf{b}$, write $\|\mathbf{b}\|$ as its Euclidean norm. For string a , use $\|a\|$ to denote the length of a . For $b \in \mathbb{Z}$, function $\text{bin}_k(b)$ converts b to a binary string a of length k (padding 0 at leading locations if necessary), and $\text{dec}_k(a)$ denotes the inverse of $\text{bin}_k(b)$. For $\mathbf{v} = (v_1, \dots, v_n)^\top \in \mathbb{Z}_q^n$, define $\text{bin}_{\lceil \log q \rceil}(\mathbf{v}) = (\text{bin}_{\lceil \log q \rceil}(v_1)^\top \parallel \dots \parallel \text{bin}_{\lceil \log q \rceil}(v_n)^\top)^\top \in \{0, 1\}^{n \lceil \log q \rceil}$.

For a finite set $\mathcal{S}$, denote by $x \xleftarrow{\$} \mathcal{S}$ sampling x uniformly from $\mathcal{S}$, and by $y \leftarrow D$ sampling y according to distribution D . Use λ to denote the security parameter throughout the paper.

2.1. Background on lattices

Let $q > 2, n, m \in \mathbb{Z}^+$. For $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, define two lattices as $\Lambda^\perp(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A} \cdot \mathbf{x} = \mathbf{0} \pmod{q}\}$ and $\Lambda^\mathbf{u}(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A} \cdot \mathbf{x} = \mathbf{u} \pmod{q}\}$. Use $D_{L,\sigma}$ to denote a discrete Gaussian distribution over lattice L with parameter $\sigma > 0$. Below we recall the definitions and hardness results of SIS, LWE and BLWE, which are needed for security proof of our scheme.

Definition 1 (SIS). Given parameters m, q, β the functions of n and uniformly random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\text{SIS}_{n,m,q,\beta}$ demands to find a non-zero vector $\mathbf{x} \in \Lambda^\perp(\mathbf{A})$ such that $\|\mathbf{x}\| \leq \beta$.

The hardness of $\text{SIS}_{n,m,q,\beta}$ is given by a worst-case to average-case reduction from standard lattice problems, such as SIVP_γ for some $\gamma = \beta \cdot \tilde{O}(\sqrt{nm})$ [1,16]. Assume m is large enough, hardness of SIS usually depends only on n, q, β , thus hereafter we write $\text{SIS}_{n,m,q,\beta}$ as $\text{SIS}_{n,q,\beta}$ for short (so as for LWE and BLWE).

Definition 2 (LWE). Let $n, m \geq 1, q \geq 2$, and Let χ be a probability distribution over $\mathbb{Z}$. For $\mathbf{s} \in \mathbb{Z}_q^n$, let $A_{\mathbf{s},\chi}$ be the distribution obtained by sampling $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$ and $e \leftarrow \chi$, and outputting the pair $(\mathbf{a}, \mathbf{a}^\top \cdot \mathbf{s} + e) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$. The $\text{LWE}_{n,q,\chi}$ problem is to distinguish m samples from $A_{\mathbf{s},\chi}$ (let $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$) and m samples chosen according to the uniform distribution over $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

For prime power $q, \beta \geq \sqrt{n}O(\log n), \gamma = \tilde{O}(nq/\beta)$, and a β -bounded distribution χ (i.e., $\chi = D_{\mathbb{Z}_m,\sigma}$), the $\text{LWE}_{n,q,\chi}$ problem is at least as hard as SIVP_γ [9,42].

Definition 3 (BLWE). Given parameters m, q the functions of n and $\chi = \{0, 1\}$ a probability distribution. For $\mathbf{s} \in \{0, 1\}^n$, let $A_{\mathbf{s},\chi}$ be the distribution generated by sampling $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$ and $e \leftarrow \chi$, and outputting the pair $(\mathbf{a}, \mathbf{a}^\top \cdot \mathbf{s} + e) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$. The $\text{BLWE}_{n,q}$ problem is to distinguish m samples from $A_{\mathbf{s},\chi}$ and m samples chosen according to the uniform distribution over $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

For integers $n, m = n(1 + \Omega(1/\log n))$ and prime modulus $q \geq n^{O(1)}$, the $\text{BLWE}_{n,q}$ problem is at least as hard as some approximating lattice problems within a factor $\gamma = \tilde{O}(\sqrt{n}q)$ [35].

2.2. Dynamic lattice-based merkle-tree accumulators

Ling et al. [31] upgraded the static LNWX accumulator [28] with an efficient updating algorithm. The accumulator is constructed on a hash-function family $\mathcal{H}_a = \{h_{\mathbf{A}} \mid \mathbf{A} = [\mathbf{A}_0 \mid \mathbf{A}_1] \in \mathbb{Z}_q^{n \times m}\}$ that hash any binary string pair $(\mathbf{u}_0, \mathbf{u}_1)$ into $h_{\mathbf{A}}(\mathbf{u}_0, \mathbf{u}_1) = \text{bin}_{nk}(\mathbf{A}_0 \cdot \mathbf{u}_0 + \mathbf{A}_1 \cdot \mathbf{u}_1 \pmod{q}) \in \{0, 1\}^{nk}$. Formally, the updatable accumulator with $N = 2^\ell$ leaves consists of five algorithms.

- **TSetup**(1^λ): Sample $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$.
- **TAcc**($R = \{\mathbf{d}_0 \in \{0, 1\}^{nk}, \dots, \mathbf{d}_{N-1} \in \{0, 1\}^{nk}\}$): $\forall j \in [0, N-1]$, let $\mathbf{d}_j = \mathbf{u}_{j_1, \dots, j_\ell}$ where $\text{bin}_\ell(j) = (j_1, \dots, j_\ell) \in \{0, 1\}^\ell$. At depth $i \in [\ell]$, $\forall (b_1, \dots, b_i) \in \{0, 1\}^i$, node $\mathbf{u}_{b_1, \dots, b_i}$ is defined as $h_{\mathbf{A}}(\mathbf{u}_{b_1, \dots, b_i, 0}, \mathbf{u}_{b_1, \dots, b_i, 1})$, and root $\mathbf{u}$ is defined as $h_{\mathbf{A}}(\mathbf{u}_0, \mathbf{u}_1)$.
- **TWitness**($R, \mathbf{d}$): Return $\perp$ if $\mathbf{d} \notin R$. Else, output witness of $\mathbf{d}$ as: $w = ((j_1, \dots, j_\ell), (\mathbf{u}_{j_1, \dots, j_{\ell-1}}, \dots, \mathbf{u}_{j_1, \bar{j}_2}, \mathbf{u}_{\bar{j}_1})) \in \{0, 1\}^\ell \times (\{0, 1\}^{nk})^\ell$, where $\bar{b}$ denotes the bit $1 - b$.
- **TVerify**($\mathbf{u}, \mathbf{d}, w$): Recursively compute the path $\mathbf{v}_\ell, \mathbf{v}_{\ell-1}, \dots, \mathbf{v}_0 \in \{0, 1\}^{nk}$. First set $\mathbf{v}_\ell = \mathbf{d}$, and $\forall i \in \{\ell-1, \dots, 0\}$, $\mathbf{v}_i = \bar{j}_{i+1} \cdot h_{\mathbf{A}}(\mathbf{v}_{i+1}, \mathbf{w}_{i+1}) + j_{i+1} \cdot h_{\mathbf{A}}(\mathbf{w}_{i+1}, \mathbf{v}_{i+1})$. Then return 1 if $\mathbf{v}_0 = \mathbf{u}$ and 0 otherwise.
- **TUpdate**($(j_1, \dots, j_\ell), \mathbf{d}^*$): Set $\mathbf{v}_\ell = \mathbf{d}^*$ and compute $\mathbf{v}_\ell, \mathbf{v}_{\ell-1}, \dots, \mathbf{v}_0$ using the same formula above. Then update the associated nodes as $\mathbf{u} = \mathbf{v}_0, \mathbf{u}_{j_1} = \mathbf{v}_1, \dots, \mathbf{u}_{j_1, \dots, j_\ell} = \mathbf{v}_\ell = \mathbf{d}^*$.

The above accumulator is proved to be secure in [31], in the sense that a value $\mathbf{d}^*$ cannot pass the verification of accumulating into the root if it is not a tree node, under the hardness of SIS.

2.3. Lattice-based public key encryption

Lindner et al. [29] constructed a CPA secure public key encryption whose security is based on the LWE assumption. Let $n_1, n_2, q, \ell, \sigma$ be positive integers. The scheme is recalled as follows:

- **PKen**(1^λ): Sample $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_q^{n_1 \times n_2}$, $\mathbf{S} \leftarrow D_{\sigma}^{\ell \times n_1}$, and $\mathbf{E} \leftarrow D_{\sigma}^{\ell \times n_2}$. Then compute $\mathbf{P} = \mathbf{S} \cdot \mathbf{B} + \mathbf{E}$. Output the public key $(\mathbf{B}, \mathbf{P})$ and secret key $\mathbf{S}$.

- $\text{PEnc}(\mathbf{B}, \mathbf{P}, \mathbf{m})$: To encrypt message $\mathbf{m} \in \{0, 1\}^\ell$, first sample $\mathbf{r} \leftarrow D_\sigma^n$, $\mathbf{e}_1 \leftarrow D_\sigma^n$, and $\mathbf{e}_2 \leftarrow D_\sigma^n$. Then compute $\mathbf{c}_1 = \mathbf{B} \cdot \mathbf{r} + \mathbf{e}_1$, $\mathbf{c}_2 = \mathbf{P} \cdot \mathbf{r} + \mathbf{e}_2 + \lfloor \frac{q}{2} \rfloor \cdot \mathbf{m}$. The ciphertext is $(\mathbf{c}_1, \mathbf{c}_2)$.
- $\text{PDec}(\mathbf{S}, \mathbf{c}_1, \mathbf{c}_2)$: Compute $\mathbf{m}' = \mathbf{c}_2 - \mathbf{S} \cdot \mathbf{c}_1$. Then $\forall j \in [\ell]$, it sets $m_j = 0$ if $\|\mathbf{m}'[j]\| < \lfloor \frac{q}{4} \rfloor$ and $m_j = 1$ otherwise. Finally output $(m_1, \dots, m_\ell)^\top$.

In this paper, we use a binary variant of this scheme. Namely, entries of $\mathbf{S}, \mathbf{E}, \mathbf{r}, \mathbf{e}$ are samples from binary set $\{0, 1\}$ instead of Gaussian distribution. Security of this variant relies on the BLWE assumption.

2.4. Zero-knowledge arguments of knowledge

In a ZKAoK system [10], a prover proves the possession of some witness for an NP relation to a verifier, without revealing any additional information. Generally, a secure ZKAoK must satisfy three requirements: *completeness*, *proof of knowledge* and (*honest or dishonest verifier*) *zero-knowledge* [17].

One classic type of ZKAoK is Stern-like protocol, which was originally proposed in code-based context [43], and then widely developed under the lattice setting for privacy-preserving schemes [26–28,31]. Most of them consist of 3 moves: three commitments cmt binding to an NP secret string, one challenge ch from space $\{1, 2, 3\}$ and responses used to check 2 of 3 commitments according to the value of ch . Such that all have inherent soundness error $2/3$, and are always repeated $\text{poly}(\lambda)$ times and then made non-interactive via Fiat-Shamir heuristic.

Recently Yang et al. [50] proposed a more efficient lattice-based ZKAoK for relation

$$\mathcal{R} = \{(\mathbf{M}, \mathbf{y}, \mathcal{L}), (\mathbf{x}) : \mathbf{M} \cdot \mathbf{x} = \mathbf{y} \wedge \forall (i, j, k) \in \mathcal{L}, \mathbf{x}[i] = \mathbf{x}[j] \cdot \mathbf{x}[k]\}, \quad (1)$$

where $\mathbf{x} \in \mathbb{Z}_q^n$ is the secret witness and $\mathcal{L}$ is a set of triples defining quadratic constraints over $\mathbf{x}$ whose size is l . The protocol can then be transformed into an NIZKAoK consisting of two algorithms *Prove* and *Verify* via Fiat-Shamir heuristic. Given parameters $(\mathcal{G}, l_1, l_2, p, \kappa, t, \sigma_1, \sigma_2)$ defined in [50, Sec. 3.1], *Prove* produces commitments $cmt = (\overline{cmt}, \overline{cmt})$, a challenge $ch = \mathcal{G}(cmt, \cdot)$ where $\mathcal{G}$ is modeled as a random oracle, and some responses rsp via ch and $\mathbf{x}$. Then it dispatches $(\overline{cmt}, ch, rsp)$ to a verifier, having $(\log(2p+1) + \kappa + (3l_1 + 2l_2 + 2n + 2l) \cdot k) \cdot t + (l_1 + n) \cdot k$ bits. *Verify* uses the three received components to recover the reserved commitments $\overline{cmt}$, then computes ch' by recovering the original cmt and checks if it is equal to the original value ch . We utilize this NIZKAoK to build our SH scheme and refer readers to Figure 2 of [50] for more detailed descriptions.

Theorem 1. *The protocol described in Figure 2 of [50] is a secure NIZKAoK having negligible completeness and soundness error, under the hardness assumptions of SIS and LWE, and has a well-designed simulator and knowledge extractor.*

3. Model of fully dynamic secret handshake

Based on the definitions in [5,12], FDSH involves the following entities: group authority GA that manages member enrollment and revocation; tracing authority TA that can identify handshake transcripts; and group users that can anonymously authenticate each other. GA interacts with a newcomer to register her into a group, where the user is issued a group credential and keeps a user secret herself. GA also publishes group information info_τ with a unique tag τ (referred to as time epoch hereafter) when there is a member change. In this way, by downloading info_τ , users can autonomously update their credentials. Formally, let λ denote the security parameter, FDSH consists of the following polynomial-time algorithms:

- **Setup**: On input λ , this algorithm generates public parameters par common to all the groups.
- **KGen_{TA}**: On input par , this algorithm is invoked by TA to output a tracing key pair (tpk, tsk) . For efficient management, we assume all the groups are supervised by the same tracing keys.
- **CreateGroup**: On input par , this algorithm is run by GA to establish a new group. Namely, it outputs a manager key pair (mpk, msk) . Meanwhile, it outputs the initial empty group information info . Note that group public key $\text{gpk} = (\text{mpk}, \text{tpk})$ and group information info are published by GA.
- **AddMember**: This is an interactive algorithm run by a user U and GA to conduct user enrollment. Concretely, at epoch τ_u :
 - On input par and gpk , it outputs user's key pair (usk, upk) for U . Then U sends upk to GA.
 - On input par , gpk , msk , plus the received upk , GA runs this algorithm to output a dynamic group credential cred (including group identity ID) for U and the updated info_{τ_u} . Then GA sends cred to U and publishes the new info_{τ_u} .
- **Handshake**: This is a mutual authentication algorithm executed by two members A and B . After several flows of interactions At epoch τ_h , it outputs 1 for both parties if and only if A and B belong to the same group (i.e., possess valid¹ credentials of the same group). Otherwise it outputs 0 for A and B .

¹ We call a credential valid if and only if it is legal (issued by GA) and active (not revoked) at current epoch.

Remark 1. One additional desirable property for Handshake is to generate the same session key for both participants if they authenticate each other successfully. For generality, we do not mention this notion in the model of FDSH but will achieve it within our instantiation in Section 5.

- **TraceMember:** On input a handshake transcript T and tracing keys (tpk, tsk) , this algorithm is invoked by TA to output ID of the involved user related to T , and a proof π_{trace} arguing the tracing result is correct; or it outputs $\perp$ to indicate a failure.
- **JudgeMember:** On input triple $(T, \text{ID}, \pi_{\text{trace}})$, this algorithm is invoked by other group members to check the validity of tracing result, outputting 1 if it is the case or 0 otherwise.
- **RemoveMember:** At epoch τ_r , on input par , gpk , msk , and a credential cred to be revoked, this algorithm is run by GA to remove the corresponding user from her group. It outputs the updated group information info_{τ_r} , which is then published by GA.

Following the considerations in [5,24], we carefully reform the security requirements of FDSH as *FD-completeness*, *FD-impersonator resistance*, *FD-detector resistance*, *FD-unlinkability* and *FD-traceability*, all of which are defined via the corresponding experiments. Hereafter denote the corruption list of users and groups by CU and CG, respectively.

FD-completeness demands that Handshake outputs 1 with overwhelming probability if both participants belong to the same group. Moreover, TraceMember can always identify the involved user and generate a valid proof. For plain description, we define an auxiliary polynomial-time algorithm as follows,

$\text{IsActive}(\text{ID}, \tau)$: Outputs 1 if the user is active at epoch τ or 0 otherwise.

Definition 4. FD-completeness is achieved if the experiment below returns 1 with negligible probability.

Experiment: $\text{Exp}_{\mathcal{A}}^{\text{COM}}(\lambda)$

$\text{par} \leftarrow \text{Setup}(\lambda)$; $(\text{info}, \text{mpk}, \text{msk}, \text{tpk}, \text{tsk}) \leftarrow (\text{KGen}_{\text{TA}}(\text{par}), \text{CreateGroup}(\text{par}))$; $\text{gpk} := (\text{mpk}, \text{tpk})$.
 $\{(\text{ID}_0, \text{cred}_0), (\text{ID}_1, \text{cred}_1)\} \leftarrow \text{AddMember}(\text{GA}, \cdot)$; $\forall b \in \{0, 1\}, \text{IsActive}(\text{ID}_b, \tau) = 1$.
 If $\text{Handshake}(\text{ID}_0, \text{ID}_1, \tau) = 0$ with transcript T , return 1.
 $\{\text{ID}, \pi_{\text{trace}}\} \leftarrow \text{TraceMember}(T, \text{tpk}, \text{tsk})$, if $\text{ID} \notin \{\text{ID}_0, \text{ID}_1\} \vee \text{JudgeMember}(T, \text{ID}, \pi_{\text{trace}}) = 0$, return 1.
 Else return 0.

FD-impersonator resistance requires that it is infeasible for any adversary $\mathcal{A}$ to impersonate a valid member of an uncorrupted group, even if it can corrupt all the users and groups except the chosen pair and can take control of TA via accessing the following oracles.

- $\text{CorG}(G)$ returns msk of some group G to $\mathcal{A}$ and adds G to CG, meaning that G is under the control of $\mathcal{A}$, e.g., $\mathcal{A}$ can obtain or modify the credential of any legal user of G .
- $\text{CorT}(\text{par})$ returns tsk of TA to $\mathcal{A}$, indicating that TA is controlled by $\mathcal{A}$.
- $\text{CorU}(\text{ID}, G)$ returns $(\text{cred}, \text{usk})$ of user ID in G to $\mathcal{A}$ and adds (ID, G) to CU, meaning that ID is under the control of $\mathcal{A}$, such that $\mathcal{A}$ can conduct a successful handshake with any other valid user in G .
- $\text{Add}(U, G)$ enrolls user U to G , whose key pair (upk, usk) is chosen by $\mathcal{A}$, meaning that $\mathcal{A}$ can obtain the issued credential of U . Then it adds (ID_U, G) to CU and updates info_{τ} at current epoch τ .
- $\text{HS}(U)$ simulates an uncorrupted user U to handshake with $\mathcal{A}$, it outputs the overall transcripts during the interaction by use of U 's credential from the internal state.
- $\text{Remove}(\text{ID}, G)$ allows $\mathcal{A}$ to revoke user ID from G and update info_{τ} at current epoch τ .
- $\text{Chal}^{\text{IR}}(\tau, \text{ID}, G)$ simulates ID of G to Handshake with $\mathcal{A}$ at epoch τ , and returns the corresponding output (0/1).

Observe that oracles CorU and Add simulate two ascendant user corruption scenarios, where the latter endows $\mathcal{A}$ with more power to create dummy users or perform injection attacks. Besides, $\mathcal{A}$ can also start HS (though the output will always be 0) or query Remove within users of uncorrupted groups, to collect desired group data.

Definition 5. FD-impersonator resistance is achieved if, for any PPT $\mathcal{A}$, the experiment below returns 1 with negligible probability.

Experiment: $\text{Exp}_{\mathcal{A}}^{\text{IR}}(\lambda)$

$\text{par} \leftarrow \text{Setup}(\lambda)$; $\text{CG}, \text{CU} := \emptyset$; $(\text{info}, \text{mpk}, \text{tpk}) \leftarrow (\text{KGen}_{\text{TA}}(\text{par}), \text{CreateGroup}(\text{par}))$; $\text{gpk} := (\text{mpk}, \text{tpk})$.
 $(\tau^*, \text{ID}^*, G^*) \leftarrow \mathcal{A}^{(\text{CorG}, \text{CorU}, \text{Add})(\cdot, \cdot, G^*), \text{CorT}, \text{HS}, \text{Remove}}(\text{par}, \text{gpk})$.
 Return 1 if $\text{Chal}^{\text{IR}}(\tau^*, \text{ID}^*, G^*) = 1 \wedge G^* \notin \text{CG} \wedge (\cdot, G^*) \notin \text{CU}$. Else return 0.

FD-detector resistance makes sure that an adversary cannot succeed when activating a handshake protocol with an uncorrupted user to identify her affiliation at the chosen epoch, even if it can corrupt all the users and groups except the chosen pair, via the following extra oracles.

- $\text{Trace}(T, \tau)$ returns the identities of the participants in T together with a proof, with respect to the epoch τ . Hereafter we require that T is not generated from the related challenging oracles.
- $\text{Chal}_b^{DR}(\tau, \text{ID}, G)$ chooses a random bit $b \in \{0, 1\}$. For $b = 0$, it simulates ID of G to handshake with $\mathcal{A}$. For $b = 1$, it simulates an arbitrary legal user ID_r (not in group G) to handshake with $\mathcal{A}$.

Definition 6. FD-detector resistance is achieved if, for any PPT $\mathcal{A}$, the absolute difference of probability of outputting 1 between experiment $\text{Exp}_{\mathcal{A}}^{DR-1}$ and $\text{Exp}_{\mathcal{A}}^{DR-0}$ is negligible.

Experiment: $\text{Exp}_{\mathcal{A}}^{DR-b}(\lambda)$

$\text{par} \leftarrow \text{Setup}(\lambda)$; $\text{CG}, \text{CU} := \emptyset$; $(\text{info}, \text{mpk}, \text{tpk}) \leftarrow (\text{KGen}_{\text{TA}}(\text{par}), \text{CreateGroup}(\text{par}))$; $\text{gpk} := (\text{mpk}, \text{tpk})$.
 $(\tau^*, \text{ID}^*, G^*) \leftarrow \mathcal{A}(\text{CorG}, \text{CorU}, \text{Add})(\cdot, \neg G^*), \text{HS}, \text{Remove}, \text{Trace}(\text{par}, \text{gpk})$, holding that $G^* \notin \text{CG} \wedge (\cdot, G^*) \notin \text{CU}$.
 $b^* \leftarrow \mathcal{A}^{\text{Chal}_b^{DR}}(\tau^*, \text{ID}^*, G^*)(\text{par}, \text{gpk})$. Return b^* .

Note that in this experiment, $\mathcal{A}$ cannot corrupt TA while it can access a restricted tracing oracle Trace .

FD-unlinkability ensures that no adversary can distinguish whether two executions of Handshake involve the same honest user, even if it can corrupt any group and any user including the chosen ones. The corresponding challenging oracle is depicted as follows.

- $\text{Chal}_b^{\text{Unlink}}(\tau, \text{ID}_0, \text{ID}_1, G^*)$ picks a random bit $b \in \{0, 1\}$, and successionaly simulates ID_0 and ID_b (both are from group G^*) to handshake with $\mathcal{A}$.

Definition 7. FD-unlinkability is achieved if, for any PPT $\mathcal{A}$, the absolute difference of probability of outputting 1 between experiment $\text{Exp}_{\mathcal{A}}^{\text{Unlink}-1}$ and $\text{Exp}_{\mathcal{A}}^{\text{Unlink}-0}$ is negligible.

Experiment: $\text{Exp}_{\mathcal{A}}^{\text{Unlink}-b}(\lambda)$

$\text{par} \leftarrow \text{Setup}(\lambda)$; $\text{CoG}, \text{CoU} := \emptyset$; $(\text{info}, \text{mpk}, \text{tpk}) \leftarrow (\text{KGen}_{\text{TA}}(\text{par}), \text{CreateGroup}(\text{par}))$; $\text{gpk} := (\text{mpk}, \text{tpk})$.
 $(\tau^*, \text{ID}_0, \text{ID}_1, G^*) \leftarrow \mathcal{A}^{\text{CorG}, \text{CorU}, \text{Add}, \text{HS}, \text{Remove}, \text{Trace}}(\text{par}, \text{gpk})$.
 $b^* \leftarrow \mathcal{A}^{\text{Chal}_b^{\text{Unlink}}}(\tau^*, \text{ID}_0, \text{ID}_1, G^*)(\text{par}, \text{gpk})$. Return b^* .

FD-traceability asks that $\mathcal{A}$ is unable to execute a successful handshake with an honest user, such that the tracing result of TA is either an uncorrupted user or a failure, even if it can corrupt any user, and is given access to Trace . Note that here $\mathcal{A}$ cannot corrupt any group, i.e., cannot query CorG . We stress that this property guarantees that no user could get off scot-free if she has successfully authenticated others, compared with *FD-completeness* which only ensures the behaviors of honest users to be traceable.

Definition 8. FD-traceability is achieved if the experiment below returns 1 with negligible probability.

Experiment: $\text{Exp}_{\mathcal{A}}^{\text{Trace}}(\lambda)$

$\text{par} \leftarrow \text{Setup}(\lambda)$; $\text{CoG}, \text{CoU} := \emptyset$; $(\text{info}, \text{mpk}, \text{tpk}) \leftarrow (\text{KGen}_{\text{TA}}(\text{par}), \text{CreateGroup}(\text{par}))$; $\text{gpk} := (\text{mpk}, \text{tpk})$.
 $(\text{ID}^*, G^*, \text{info}_{\tau^*}) \leftarrow \mathcal{A}^{\text{CorU}, \text{Add}, \text{HS}, \text{Remove}, \text{Trace}}(\text{par}, \text{gpk})$,
If $\text{Handshake}(\mathcal{A}, \text{ID}^*, \tau^*) = 0$ with transcript T , return 0.
 $(\text{ID}, \pi_{\text{trace}}) \leftarrow \text{TraceMember}(T, \text{tpk}, \text{tsk})$.
If $(\text{ID}, G^*) \notin \text{CU} \vee (\text{ID}, \pi_{\text{trace}}) = \perp \vee \text{JudgeMember}(T, \text{ID}, \pi_{\text{trace}}) = 0$, return 1. Else return 0.

Full Deniability. Now we consider *full deniability* proposed in [44]. Informally, this property demands that for any group in SH, no PPT $\mathcal{A}$ can provide a convincing proof (record) to a third party, showing that some legal user of that group has participated in a conversation, even if it can monitor the public system channel or actually perform handshakes with other users.

Following previous work in deniable authentication [40,41,46], [44] presents a simulator-based definition of *full deniability*. Let Aux denote the sets of all public data (including $\text{par}, \text{gpk}, \text{tpk}, \text{info}, \pi_{\text{trace}}$ and transmitted texts between users) accessible to $\mathcal{A}$ via C , $\text{View}_{\mathcal{A}}(\text{Aux})$ denote the real view of $\mathcal{A}$ when it collects transcripts of finished handshakes or performs Handshake (by querying CorG, CorU or Add) with other honest users. Below we modify the definition in [44] to fit the fully dynamic setting.

Definition 9. An FDSH scheme is fully deniable if for any PPT $\mathcal{A}$ and auxiliary $\text{aux} \in \text{AUX}$, there exists a PPT simulator S , which, feeding with aux , can produce a simulated view $\text{View}_S(\text{aux})$, such that for the two probability distributions $\text{Real}(\lambda, \text{aux}) = [\text{aux}, \text{View}_{\mathcal{A}}(\text{aux})]$, $\text{Sim}(\lambda, \text{aux}) = [\text{aux}, \text{View}_S(\text{aux})]$, and for all PPT distinguishers Dst , it holds that

$$\Pr[x \in \text{Real}(\lambda, aux) : \text{Dst}(x) = 1] - \Pr[x \in \text{sim}(\lambda, aux) : \text{Dst}(x) = 1] \leq \epsilon,$$

where ϵ is a negligible function of security parameter λ , and Dst is fed all the group secrets (including group and user secret keys) except for the tracing secret key.

Note that here $\mathcal{A}$ can corrupt any group, that is, even malicious GA cannot break *full deniability*.

4. Anonymous message exchange

In this section, first we give some motivations for why we introduce this new primitive and how it can simplify the design of SH. Then formal definitions of AME and the associated security requirements are presented. Further, two generic constructions from NIZKAoK systems are proposed. Finally, a concrete lattice-based instantiation of AME is built to serve as Handshake of our FDSH scheme.

4.1. Motivations

Our starting point is that in SH, the core algorithm Handshake is run by two participants (A, B) to anonymously authenticate each other and negotiate a session key. Most of the previous SH schemes [5,12,24,47,23,51,3] capture this functionality via the same technical way, i.e., A and B send each other some credential-related texts, from which they both “extract” (compute) a KE element (e.g., Diffie-Hellman-like one), such that they can negotiate a session key and authenticate each other via a hash code generated from that session key. A two-round sketch of this line is as follows:

1. $A \rightarrow B : (\Sigma_a)$
 A picks a Diffie-Hellman key pair (a, g^a) and uses her group credential to generate a text Σ_a , where g^a is embedded.
2. $B \rightarrow A : (\Sigma_b, \mathbb{V}_b)$
 B also picks an ephemeral key pair (b, g^b) and uses his group credential to embed g^b into a text Σ_b . Then B recovers g^a from Σ_a to compute the session key $\kappa = (g^a)^b$, the hash code can be simply set as $\mathbb{V}_b = \mathcal{H}(\kappa, g^a, \cdot)$ where $\mathcal{H}$ is a secure hash function.
3. $B \rightarrow A : (\mathbb{V}_a)$
Similarly, A recovers g^b from Σ_b to compute the session key $\kappa = (g^b)^a$, and sets $\mathbb{V}_a = \mathcal{H}(\kappa, g^b, \cdot)$. Finally, A and B verify the received hash tags to conduct mutual authentication.

From the above abstract description, we observe that the available design of Handshake can be broken down into two ordinal sub-steps: 1). enable legal users to exchange some random message which is hidden by and bonded to their group credentials; 2). leverage suitable KE protocols to negotiate a session key.

The second goal is easy to realize since there have already been secure and efficient KE protocols for different system scenarios. On the other side, the first task is always challenging, due to the coupled demand of offering user's anonymity and interaction's validity. All the previous work [5,12,24,47,23,51,3] have implicitly achieved step 1) via various cryptographic techniques, e.g., RSA encryptions, oblivious transfer, group signatures or (code/lattice)-based ZK proofs. In a sum, Handshake (i.e., anonymous mutual authentication) can be directly obtained if step 1) is realized. To this extent, one may wish that there are some generic way of reducing the design of Handshake to that of step 1), such that the overall construction of an SH scheme can be modularized and simplified. However, no attempts have been made to explore theoretical grasp of step 1), let alone give a feasible framework. To fill this gap, we distill the intrinsic functionality that step 1) requires and formalize it as a new primitive: anonymous message exchange.

4.2. Syntax

Extended from the aforementioned step 1), an AME scheme consists of three algorithms *Setup*, *Send* and *Open*. *Setup* generates system public/secret keys, along with all the user's credentials.² *Send* is invoked by users to anonymously dispatch a text Σ with a hidden message M embedded in it, and anyone can run *Open* to retrieve a potential message M' , whose correctness ($M' = M$) depends on whether the sender has a valid credential. Informally, AME endows participants to recover some message from others' public texts while they cannot verify it, yet the correctness of the retrieved message relies on whether the sender is legitimate. The formal definition of AME is as follows.

Definition 10. An AME scheme consists of 3 polynomial-time algorithms:

- $\text{Setup}(\lambda) \rightarrow (\text{gpk}, \text{gsk}, \text{cred})$. With security parameter λ , this algorithm generates system public/secret keys (gpk, gsk) , along with user's credentials cred . W.L.O.G, assume that there is a trusted authority (e.g., CA in PKI) to conduct this

² For simplicity, an AME system is static by default, and dynamicity is considered when building our main SH scheme.

procedure, and no one else can produce a valid credential without gsk (e.g., cred belongs to some NP-hard relation set $\mathcal{LC}$ with trapdoor gsk).

- $\text{Send}(\text{gpk}, \text{cred}, M) \rightarrow \Sigma$. With gpk, cred and a message M from message space $\mathcal{M}$, this PPT non-deterministic algorithm generates a text Σ .
- $\text{Open}(\text{gpk}, \Sigma) \rightarrow M$. This algorithm recovers the hidden message M within input Σ using gpk.

Remark 2. Note that AME does not guarantee the secrecy of the hidden message, that is, anyone with gpk can get M from Σ . On the other hand, we place restrictions on who can produce an available text via Send, i.e., whether the recovered M' is identical to M hinges on whether the sender has a valid credential.

Below we give the correctness and the two security requirements of AME.

Correctness asks that one can always run Open to recover the correct message from a text generated by Send fed with a valid credential and message, i.e., let $\text{gpk} \leftarrow \text{Setup}(\lambda)$, $\forall \text{cred} \in \mathcal{LC}$, $M \in \mathcal{M}$, and $\Sigma \leftarrow \text{Send}(\text{gpk}, \text{cred}, M)$, it holds that $\Pr[\text{Open}(\text{gpk}, \Sigma) \neq M] = \text{negl}(\lambda)$.

Soundness ensures that for some chosen message M^* , no one can forge an available text Σ^* without a valid credential, i.e., M^* can be correctly recovered from Σ^* by Open. In the following soundness game, $\mathcal{A}$ is not given access to either gsk or cred.

- Start. The challenger runs $\text{Setup}(\lambda)$ to generate $(\text{gpk}, \text{gsk}, \text{cred})$, then gives gpk to $\mathcal{A}$.
- Queries. $\mathcal{A}$ can make the following queries:
 - Query for valid text of any legal user d on any message $M \in \mathcal{M}$. The challenger returns the text $\Sigma = \text{Send}(\text{gpk}, \text{cred}_d, M)$.
- Challenge. The challenger chooses a random message $M^* \in \mathcal{M}$ and returns M^* to $\mathcal{A}$.
- Restricted Queries. After the challenge phase, $\mathcal{A}$ can still make queries as before, but with the following restriction: it is not allowed to make any query for message M^* .
- Forgery. Finally, $\mathcal{A}$ outputs a text Σ^* , it wins the game if $\text{Open}(\text{gpk}, \Sigma^*) = M^*$.

The probability that $\mathcal{A}$ wins the game, denoted by $\text{SuccPS}_{\mathcal{A}}$, is taken over the randomness of $\mathcal{A}$, Setup and Send. We say that an AME scheme maintains soundness if $\text{SuccPS}_{\mathcal{A}}$ is negligible.

Full Anonymity demands that no one can determine which of the two adaptively chosen users generated a text. Now in the following full anonymity game, $\mathcal{A}$ is given access to both gsk and cred.

- Start. The challenger runs $\text{Setup}(\lambda)$ to generate $(\text{gpk}, \text{gsk}, \text{cred})$, then gives (gpk, gsk) to $\mathcal{A}$.
- Queries. $\mathcal{A}$ can make the following queries:
 - Query for valid text of any legal user d on any message $M \in \mathcal{M}$. The challenger returns the text $\Sigma = \text{Send}(\text{gpk}, \text{cred}_d, M)$.
 - Query for credential of any legal user d . The challenger returns cred_d .
- Challenge. $\mathcal{A}$ outputs a message M^* and two users d_0 and d_1 , note that these users may have already been corrupted. The challenger chooses a bit $b \xleftarrow{\$} \{0, 1\}$, computes a text of user d_b on M^* as $\Sigma^* = \text{Send}(\text{gpk}, \text{cred}_{d_b}, M^*)$, and returns Σ^* to $\mathcal{A}$.
- Adaptive Queries. After the challenge phase, $\mathcal{A}$ can still make queries as before without any restrictions.
- Output. Eventually, $\mathcal{A}$ outputs a bit b' . It wins the game if $b' = b$.

We define the adversary's advantage in winning the game as $\text{Adv}_{\mathcal{A}} = |\Pr[b' = b] - \frac{1}{2}|$. We say that an AME scheme achieves fully anonymous if $\text{Adv}_{\mathcal{A}}$ is negligible.

Applications. Now we briefly introduce some immediate applications of AME to demonstrate how AME can work as an underlying module in the construction of privacy-preserving protocols. Nevertheless, to be self-contained, in this work we focus on leveraging AME to build SH schemes and leave the promising applications on other cryptographic protocols for future work.

Secret Handshake. As we mentioned at the beginning of this section, an AME scheme plus a KE protocol exactly make for Handshake of an SH scheme: participants take the public element of a KE (e.g., cyclic-group element g^x) as the hidden message M in AME, then feed Send on their group credentials to generate a handshake text Σ_{hs} , with which they can invoke Open to recover the KE element, so as to produce a session key and an authentication code. This technique road is a generic framework for Handshake, i.e., anonymous mutual authentication,³ seeing it programs the previous design [5, 12, 24, 47, 23, 51, 3]. Further, if we only consider the static model of SH, Setup of AME is theoretically equal to the launching algorithms (CreateGroup, AddMember) of SH. To better depict how AME serves as a building block of SH, we construct the first lattice-based FDSH by use of a well-designed AME scheme in the next section.

³ If negotiating a session key is not a requisite functionality, AME itself is enough to build Handshake, since authentication codes can simply be the hash outputs of any plain messages.

Group Signature with Message Recovery. [24] introduces a special type of group signature named group signature with message recovery, where the signed message need not be dispatched and can be recovered from the signature, in this way the size of a signature can be shortened. To anonymously sign a message on behalf of a group, a user could utilize algorithm *Send* to embed the message into the signature. However, to enable the signature to be verifiable, some knowledge of the original message should be generated, this can be done by use of a suitable commitment scheme, e.g., special MAC or PRF functions. To verify the succinct signature, a receiver could first invoke algorithm *Open* to retrieve the original message and then check the validity of the corresponding knowledge.

Multi-party Authentication. During a multi-party authentication (MPA) protocol [20], an AME scheme can be applied as a subprogram to broadcast authenticated public information on the system channel, meanwhile protect user's privacy to ensure anonymity. There may be some security and efficiency gain as a result of our compact framework. Further, AME-based MPA can play the role of access control for more integrated multi-party computation (MPC) systems.

4.3. Generic approaches

Here we describe how to modify two kinds of NIZKAoK termed Stern-type [13,27,26,28,30,31] and Fiat-Shamir with abort [32–34,25,14,50], to obtain generic approaches of AME, where the corresponding correctness and soundness error (c, s) will be those of the underlying system. The core idea is to send part of the commitment and modify the challenge of a ZK system, such that the whole proof cannot be directly verified by the receiver, yet the desired message can be recovered from the proof.

Note that there are natural and easy ways to build *Setup*, e.g., pairing, code, lattice-based unforgeable signatures used in [47,25,27,51,3], or any PKI programs. Thus, to give a more succinct description, we omit the content of *Setup*⁴ and only consider *Send* and *Open* in the following constructions.

Construction I: Stern-type NIZKAoK generally has 3 elements: commitments $cmt = (c_1, c_2, c_3)$, challenge $ch = \mathcal{H}(cmt, public_data) \in \{1, 2, 3\}$, and response rsp used to recover and verify 2 of 3 commitments according to the value of ch (e.g., check (c_2, c_3) for $ch = 1$). The design is as follows:

- **Send1**(gpk, cred, M): Invoke the utilized NIZKAoK to first produce a proof $\pi_1 = (cmt, \hat{ch}, rsp)$ by regarding gpk and cred as hash randomness and a secret witness, respectively, where the only change is to modify the challenge as $\hat{ch} = ch \oplus M$. Then assemble partial commitment $\overline{cmt}$, consisting of 1/3 of commitments that cannot be checked by the corresponding response (e.g., $\overline{cmt} = c_1$ for $\hat{ch} = 1$). Finally output $\Sigma = (cmt, \hat{ch}, rsp)$.
- **Open1**(gpk, Σ): Recover the reserved 2/3 of commitments $\overline{cmt}$ via rsp and get the original cmt by combining $\overline{cmt}$, so as to retrieve $M = \hat{ch} \oplus \mathcal{H}(cmt, gpk)$. Output the resulting message M .

Construction II: We first clarify some necessary points on ZKAoK termed as Fiat-Shamir with abort. The interactive (two-round) setting performs as usual: a prover P first sends a commitment cmt to a verifier V , who then picks and returns a random challenge ch , finally P responds a response rsp and V checks if cmt and rsp are ch -related consistent.

On the other hand, unlike the traditional non-interactive model where P sends $\pi = (cmt, ch, rsp)$ and V verifies if $cmt = f(ch, rsp)$ (f is a public injective map), the pioneering works [32,33] show how to subtly shorten the size of π , namely, P only sending modified $\pi = (ch, rsp)$, where $ch = \mathcal{H}(cmt, \cdot)$ and is used as a coefficient in the calculation of rsp , rather than being a selection condition as that in Stern-type NIZKAoK. In this way, V can recover cmt from π and check if $\mathcal{H}(cmt, \cdot) = ch$. Later work [25,14,50] further optimize this “computation-communication trade-off” strategy by adaptively dividing $cmt = (\overline{cmt}, \overline{cmt})$, now P turns to send $\pi = (\overline{cmt}, ch, rsp)$, such that V can recover $\overline{cmt}$ and verify if $\mathcal{H}(\overline{cmt}, \overline{cmt}, \cdot) = ch$ as well (see [25,14,50] and references therein for detailed measures). Nevertheless, we follow the aforementioned notations and now give the formal description of construction II.

- **Send2**(gpk, cred, M): Take gpk and cred as hash randomness and a witness respectively, run the underlying argument to produce commitments $cmt = (\overline{cmt}, \overline{cmt})$ and $ch = \mathcal{H}(cmt, gpk)$. Compute $\hat{ch} = ch \oplus M$ and generate rsp using $\hat{ch}$. Finally output $\Sigma = (\overline{cmt}, \hat{ch}, rsp)$.
- **Open2**(gpk, Σ): Recover $\overline{cmt}$ via rsp and $\hat{ch}$, then get the original challenge $ch = \mathcal{H}(cmt, gpk)$, so as to compute $M = \hat{ch} \oplus ch$. Output the resulting message M .

Theorem 2. The two constructions of AME are secure assuming the security of used NIZKAoK.

Proof. For *correctness*, we deduce from *completeness* of the utilized NIZKAoK that, upon receiving Σ from a legal user by running *Send*, others can always recover the right reserved commitments $\overline{cmt}$ via $\hat{ch}$ and rsp , then obtain the original cmt , so as to compute the original challenge ch with perfect probability for Stern-type NIZKAoK or negligible error rate for Fiat-Shamir with abort ones. In this way, a simple calculation $M = \hat{ch} \oplus ch$ shows that the retrieved message is always equal to the hidden one.

⁴ We present a concrete *Setup* when applying a concrete AME to build FDSH in the next section.

For *soundness* in construction II, we prove it by contradiction. Suppose that for chosen message M^* and $\Sigma^* = (\widehat{cmt}, \widehat{ch}, \widehat{rsp})$, $\mathcal{A}$ succeeds with advantage $s + \epsilon$. Then we can build a PPT algorithm $\mathcal{B}$ to break *soundness* of the underlying NIZKAoK. Specifically, $\mathcal{B}$ first recovers the reserved $\widehat{cmt}$ via $\widehat{ch}$ and $\widehat{rsp}$, then it computes the original challenge $ch = \mathcal{H}(cmt, \text{gpk})$ where $cmt = (\widehat{cmt}, \widehat{cmt})$, the fact $\mathcal{A}$ wins means that $M = \widehat{ch} \oplus ch = M^*$, which implies that ch and cmt are both identical to the original values. In this sense, $\mathcal{B}$ can assemble a valid argument $\pi^* = (cmt, ch, rsp)$ without possessing any witness. Further, $\mathcal{B}$ can run the well-designed knowledge extractor to break the computationally hard problem the NIZKAoK relies on. Case of construction I is similar to the preceding situation and we omit it for simplicity.

For *full anonymity*, since $\widehat{cmt}$ is actually part of cmt , the only difference between the output Σ and the original proof π is the challenge part. Obviously $\widehat{ch} = ch \oplus M$ is statistically indistinguishable from ch regardless of the message selection. Thus, we design the following hybrid games to prove this property:

Game 0: This is the case where the challenger picks d_0 to produce Σ_0 .

Game 1: In this game, we utilize the simulator recalled in section 2.4 to generate a proof π_{simu} , then operate the same rest lines in Send1 (Send2) to get the simulated output Σ_{simu} . By *zero-knowledge* of the NIZKAoK, we have that $\Pr[\mathcal{A}^{G^0}(\text{guess}) = 1] - \Pr[\mathcal{A}^{G^1}(\text{guess}) = 1] = \text{negl}(\lambda)$.

Game 2: Now the challenger chooses ID_1 to produce Σ_1 , where it also has $\Pr[\mathcal{A}^{G^1}(\text{guess}) = 1] - \Pr[\mathcal{A}^{G^2}(\text{guess}) = 1] = \text{negl}(\lambda)$ by the same *zero-knowledge* property of the underlying ZKAoK.

This infers that $\Pr[b = b' \leftarrow \mathcal{A}(\text{guess})] = \text{negl}(\lambda)$, which concludes the proof. $\square$

4.4. Lattice-based instantiation

Now we build a concrete AME scheme from the ZKAoK recalled in Section 2.4. Remark that Setup is actually contained within CreateGroup and AddMember in the next section. Thus our remaining task is to apply construction II on system of [50]. Since [50] only proposed a framework of NIZKAoK, we first need to establish the detailed NIZKAoK system for proving a valid user, which actually proves the overall relation of linear equations within users' secrets (group credential and user's secret key) in our FDSH scheme.

Our design makes extensive use of a number decomposition method [30], i.e., for any non-negative integer β , let $\delta_\beta = \lceil \log(\beta + 1) \rceil$ and define $\{\beta_1, \dots, \beta_{\delta_\beta}\}$ by $\beta_j = \lfloor \frac{\beta + 2^{j-1}}{2^j} \rfloor$, for $j \in [\delta_\beta]$. Then any integer $i \in [0, \beta]$ can be decomposed as $i = \mathbf{g}_{\delta_\beta} \cdot \mathbf{i}$ where $\mathbf{g}_{\delta_\beta} = (\beta_1, \dots, \beta_{\delta_\beta})$ and $\mathbf{i} \in \{0, 1\}^{\delta_\beta}$. Now given the same situation in Handshake of Section 5 with additional setting $\mathbf{j} = (j_1, \dots, j_\ell)^\top \in \{0, 1\}^\ell$, row vectors $\mathbf{g}^\top = \mathbf{1}^{nk}$ and $\mathbf{g}' = \mathbf{g}_{\delta_{nk-1}}$, the overall system can be summarized as follows:

Public input: Matrices $\mathbf{A} = [\mathbf{A}_1 | \mathbf{A}_2]$, $\mathbf{B}$, $\mathbf{P}_1$, $\mathbf{P}_2$, $\mathbf{G}$, Vectors $\mathbf{u}_\tau$, $\mathbf{c}_{1,1}$, $\mathbf{c}_{1,2}$, $\mathbf{c}_{2,1}$, $\mathbf{c}_{2,2}$, $\mathbf{k}$; Parameters par, gpk.

Prover's witness: Vectors $\mathbf{x}$, $\mathbf{p}$, $\mathbf{q}$, $\mathbf{q}'$, $\mathbf{j}$, $\mathbf{w}_\ell, \dots, \mathbf{w}_1$, $\mathbf{r}_1, \mathbf{r}_2$, $\mathbf{e}_{1,1}$, $\mathbf{e}_{1,2}$, $\mathbf{e}_{2,1}$, $\mathbf{e}_{2,2}$.

Prover's goal: Convince the verifier in zero-knowledge that the following set of modular linear equations holds:

$$\begin{cases} \mathbf{v}_\ell = \mathbf{p}, & \mathbf{A}_{1+j_i} \cdot \mathbf{v}_i + \mathbf{A}_{2-j_i} \cdot \mathbf{w}_i = \mathbf{G} \cdot \mathbf{v}_{i-1} \pmod{q}, \forall i = \ell, \ell-1, \dots, 2, \\ \mathbf{A}_{1+j_1} \cdot \mathbf{v}_1 + \mathbf{A}_{2-j_1} \cdot \mathbf{w}_1 = \mathbf{G} \cdot \mathbf{u}_\tau \pmod{q}, \\ \mathbf{A} \cdot \mathbf{x} = \mathbf{G} \cdot \mathbf{p} \pmod{q}, \\ \mathbf{p} + \mathbf{q} = \mathbf{1}^{nk}, & \mathbf{g} \cdot \mathbf{q} - \mathbf{g}' \cdot \mathbf{q}' = 0 \pmod{q}, \\ \mathbf{c}_{i,1} = \mathbf{B} \cdot \mathbf{r}_i + \mathbf{e}_{i,1}, & \mathbf{c}_{i,2} = \mathbf{P}_i \cdot \mathbf{r}_i + \mathbf{e}_{i,2} + \lfloor \frac{q}{2} \rfloor \cdot \mathbf{j}, \forall i \in \{1, 2\}. \end{cases} \quad (2)$$

To realize the above task, we are going to leverage the NIZKAoK framework [50]. Recall that the generic system [50] allows proving a linear equation $\mathcal{R}$ where the secret witness satisfies a set of quadratic constraints, i.e., relation (1) defined in Section 2.4. Therefore, our technical road is to reduce Set (2) to an instance of relation $\mathcal{R}$, then utilize the NIZKAoK system to prove this relation. Since Set (2) is somewhat complicated, we decompose it into two sub-sets, one covers the first five equations arguing the active accumulator leaf and user secret key, and the other handles the last two equations evidencing the LWE double-encryption. Towards this end, we design two sub-systems Π_1 and Π_2 reducing these two sub-sets, respectively, then build the final system Π_{hs} for the target instance of $\mathcal{R}$. The concrete constructions are as follows.

Build system Π_1 . This system covers the first five equations of Set (2). We aim to integrate them into a union relation

$$\mathcal{R}_1 = \{(\mathbf{M}_1, \mathbf{y}_1, \mathcal{L}_1), (\mathbf{x}_1) : \mathbf{M}_1 \cdot \mathbf{x}_1 = \mathbf{y}_1 \wedge \forall (i, j, k) \in \mathcal{L}_1, \mathbf{x}_1[i] = \mathbf{x}_1[j] \cdot \mathbf{x}_1[k]\}.$$

Based on the solutions in [50], we first take the following preparations:

- Let $\mathbf{v}_0 = \mathbf{u}_\tau$. For $i \in [2, \ell]$, set $\mathbf{z}_{i,0} = \mathbf{A}_1 \cdot \mathbf{v}_i + \mathbf{A}_2 \cdot \mathbf{w}_i - \mathbf{G} \cdot \mathbf{v}_{i-1}$, $\mathbf{z}_{i,1} = \mathbf{A}_1 \cdot \mathbf{w}_i + \mathbf{A}_2 \cdot \mathbf{v}_i - \mathbf{G} \cdot \mathbf{v}_{i-1}$, $\mathbf{z}'_{i,0} = (1 - j_i) \cdot \mathbf{z}_{i,0}$, $\mathbf{z}'_{i,1} = j_i \cdot \mathbf{z}_{i,1}$.
- For the root node, set $\mathbf{z}_{1,0} = \mathbf{A}_1 \cdot \mathbf{v}_1 + \mathbf{A}_2 \cdot \mathbf{w}_1$, $\mathbf{z}_{1,1} = \mathbf{A}_1 \cdot \mathbf{w}_1 + \mathbf{A}_2 \cdot \mathbf{v}_1$, $\mathbf{z}'_{1,0} = (1 - j_1) \cdot \mathbf{z}_{1,0}$, and $\mathbf{z}'_{1,1} = j_1 \cdot \mathbf{z}_{1,1}$. Besides, set $\bar{\mathbf{j}} = (1 - j_1, \dots, 1 - j_\ell)^\top$.
- Set $\hat{\mathbf{z}}_0^\top = (\mathbf{z}_{1,0}^\top \| \mathbf{z}_{2,0}^\top \| \dots \| \mathbf{z}_{\ell,0}^\top)^\top$, $\hat{\mathbf{z}}_1^\top = (\mathbf{z}_{1,1}^\top \| \mathbf{z}_{2,1}^\top \| \dots \| \mathbf{z}_{\ell,1}^\top)^\top$, $\hat{\mathbf{z}}_0^\top = (\mathbf{z}_{1,0}^\top \| \mathbf{z}_{2,0}^\top \| \dots \| \mathbf{z}_{\ell,0}^\top)^\top$, $\hat{\mathbf{z}}_1^\top = (\mathbf{z}_{1,1}^\top \| \mathbf{z}_{2,1}^\top \| \dots \| \mathbf{z}_{\ell,1}^\top)^\top$, $\hat{\mathbf{v}} = (\mathbf{v}_1^\top \| \mathbf{v}_2^\top \| \dots \| \mathbf{v}_\ell^\top)^\top$, $\hat{\mathbf{w}} = (\mathbf{w}_1^\top \| \mathbf{w}_2^\top \| \dots \| \mathbf{w}_\ell^\top)^\top$, $\hat{\mathbf{u}}_\tau = ((\mathbf{G} \cdot \mathbf{u}_\tau)^\top \| \mathbf{0}^{(\ell-1) \cdot n})^\top$.

4. Define the auxiliary matrices: $\mathbf{N}_1 = -\mathbf{I}_\ell \otimes \mathbf{A}_1$, $\mathbf{N}_2 = -\mathbf{I}_\ell \otimes \mathbf{A}_2$, and

$$\mathbf{L}_1 = \begin{pmatrix} -\mathbf{A}_1 & & & & \\ \mathbf{G} & -\mathbf{A}_1 & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & \mathbf{G} & -\mathbf{A}_1 \end{pmatrix}, \quad \mathbf{L}_2 = \begin{pmatrix} -\mathbf{A}_2 & & & & \\ \mathbf{G} & -\mathbf{A}_2 & & & \\ & & \ddots & & \\ & & & \ddots & \\ & & & & \mathbf{G} & -\mathbf{A}_2 \end{pmatrix}.$$

After the above arrangements, we can obtain the wanted variables as follows:

1. Set the public matrix $\mathbf{M}_1$ as

$$\begin{pmatrix} \mathbf{I}_\ell & \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_\ell & \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_{n\ell} & \mathbf{0} & \mathbf{L}_1 & \mathbf{N}_2 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_{n\ell} & \mathbf{L}_2 & \mathbf{N}_1 & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{A} & (-\mathbf{G}) & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I}_{nk} & \mathbf{I}_{nk} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{g} & \mathbf{g}' \end{pmatrix}, \quad (3)$$

and establish the public vector $\mathbf{y}_1 = (\mathbf{1}^\ell \ \hat{\mathbf{u}}_\tau^\top \ \mathbf{0} \ \mathbf{0} \ \mathbf{0} \ \mathbf{1}^{nk} \ \mathbf{0})^\top$.

2. The private witness $\mathbf{x}_1$ can be built as:

$$\mathbf{x}_1 = (\bar{\mathbf{j}}^\top \ \mathbf{j}^\top \ \hat{\mathbf{z}}_0^\top \ \hat{\mathbf{z}}_1^\top \ \hat{\mathbf{z}}_0^\top \ \hat{\mathbf{z}}_1^\top \ \hat{\mathbf{v}}^\top \ \hat{\mathbf{w}}^\top \ \mathbf{x}^\top \ \mathbf{p}^\top \ \mathbf{q}^\top \ \mathbf{q}'^\top)^\top.$$

3. As for the quadratic constraints over $\mathbf{x}_1$, define

$$\left\{ \begin{array}{l} \mathcal{L}_{1,1} = \{(i, i, i)\}, i \in [1, \ell]; \\ \mathcal{L}_{1,2} = \{(i, i, i)\}, i \in [2\ell + 4n\ell + 1, 2\ell + 4n\ell + 2nk\ell]; \\ \mathcal{L}_{1,3} = \{(2\ell + n \cdot (i-1) + j, 2\ell + 2n\ell + n \cdot (i-1) + j, i)\}, \\ \quad i \in [1, \ell], j \in [1, n]; \\ \mathcal{L}_{1,4} = \{(2\ell + n\ell + n \cdot (i-1) + j, 2\ell + 3n\ell + n \cdot (i-1) + j, \ell + i)\}, \\ \quad i \in [1, \ell], j \in [1, n]; \\ \mathcal{L}_{1,5} = \{(2\ell + 4n\ell + 2nk\ell + i, 2\ell + 4n\ell + 2nk\ell + i, 2\ell + 4n\ell + 2nk\ell + i)\}, \\ \quad i \in [1, 4nk + \lceil \log(nk) \rceil], \end{array} \right.$$

where $\mathcal{L}_{1,1}$ indicates that $\bar{\mathbf{j}}$ is a binary vector, $\mathcal{L}_{1,2}$ ensures $\hat{\mathbf{v}}$ and $\hat{\mathbf{w}}$ are binary vectors, $\mathcal{L}_{1,3}$ and $\mathcal{L}_{1,4}$ indicates $\mathbf{z}'_{i,0} = (1 - j_i) \cdot \mathbf{z}_{i,0}$ and $\mathbf{z}'_{i,1} = j_i \cdot \mathbf{z}_{i,1}$, $\forall i \in [1, \ell]$, respectively, and $\mathcal{L}_{1,5}$ ensures $\mathbf{x}, \mathbf{p}, \mathbf{q}, \mathbf{q}'$ are all binary vectors. Then we set $\mathcal{L}_1 = \mathcal{L}_{1,1} \cup \mathcal{L}_{1,2} \cup \mathcal{L}_{1,3} \cup \mathcal{L}_{1,4} \cup \mathcal{L}_{1,5}$.

Build system Π_2 . This system covers the last two equations. We also reduce them to a union relation

$$\mathcal{R}_2 = \{(\mathbf{M}_2, \mathbf{y}_2, \mathcal{L}_2), (\mathbf{x}_2) : \mathbf{M}_2 \cdot \mathbf{x}_2 = \mathbf{y}_2 \wedge \forall (i, j, k) \in \mathcal{L}_2, \mathbf{x}_2[i] = \mathbf{x}_2[j] \cdot \mathbf{x}_2[k]\},$$

where $\mathbf{M}_2$ is taken as

$$\begin{pmatrix} \mathbf{B} & \mathbf{I}_{n_1} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{P}_1 & \mathbf{0} & \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} & \mathbf{0} & \lceil \frac{q}{2} \rceil \cdot \mathbf{I}_\ell \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{B} & \mathbf{I}_{n_1} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{P}_2 & \mathbf{0} & \mathbf{I}_\ell & \lceil \frac{q}{2} \rceil \cdot \mathbf{I}_\ell \end{pmatrix}, \quad (4)$$

and $\mathbf{y}_2 = (\mathbf{c}_{1,1}^\top \ \mathbf{c}_{1,2}^\top \ \mathbf{c}_{2,1}^\top \ \mathbf{c}_{2,2}^\top)^\top$. Then define $\mathbf{x}_2 = (\mathbf{r}_1^\top \ \mathbf{e}_{1,1}^\top \ \mathbf{e}_{1,2}^\top \ \mathbf{r}_2^\top \ \mathbf{e}_{2,1}^\top \ \mathbf{e}_{2,2}^\top \ \mathbf{j}^\top)^\top$, and the constraint list $\mathcal{L}_2 = \{i, i, i\}, i \in [1, 2(n_1 + n_2) + 3\ell]$.

Build system Π_{hs} . This system combines Π_1 and Π_2 into one relation:

$$\mathcal{R}_{hs} = \{(\mathbf{M}_3, \mathbf{y}_3, \mathcal{L}_3), (\mathbf{x}_3) : \mathbf{M}_3 \cdot \mathbf{x}_3 = \mathbf{y}_3 \wedge \forall (h, i, j) \in \mathcal{L}_3, \mathbf{x}_3[h] = \mathbf{x}_3[i] \cdot \mathbf{x}_3[j]\}.$$

To this effect, set $\mathbf{M}_3 = \begin{pmatrix} \mathbf{M}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{M}_2 \end{pmatrix}$, $\mathbf{x}_3 = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{pmatrix}$, $\mathbf{y}_3 = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{pmatrix}$ and $\mathcal{L}_3 = \mathcal{L}_1 \cup \mathcal{L}'_2 \cup \mathcal{L}'_3$, where $\mathcal{L}'_2 = \{i, i, i\}, i \in [2\ell + 4n\ell + 2nk\ell + 4nk + \lceil \log(nk) \rceil + 1, 5\ell + 4n\ell + 2nk\ell + 4nk + \lceil \log(nk) \rceil + 2(n_1 + n_2)]$ and $\mathcal{L}'_3 = \{i, j, j\}, i \in [\ell + 1, 2\ell], j \in [4\ell + 4n\ell +$

$2nk\ell + 4nk + \lceil \log(nk) \rceil + 2(n_1 + n_2) + 1, 5\ell + 4n\ell + 2nk\ell + 4nk + \lceil \log(nk) \rceil + 2(n_1 + n_2)$ ensuring $\mathbf{j}$ in two sub-systems is the same one. Then the specific system for framework in Section 2.4 is constructed.

Obtain an AME. Finally, by applying construction II on Π_{hs} , we obtain two algorithms Send_{hs} and Open_{hs} . To further support mutual authentication, we set M as the public element in a KE [8], such that participants can compute the same session key $\mathbf{K}$ and verify each other's authentication code $\mathcal{H}(\mathbf{K}, \cdot)$. See algorithm Handshake in the next section for more details on how this lattice-based AME is applied.

5. Lattice-based FDSH with deniability

In this section, we build our FDSH by applying the above-designed AME plus suitable lattice methods. In particular, we apply the upgraded LNWX accumulators to manage dynamic user enrollment and revocation. Mutual authentication is achieved by bonding the KE [8] to the AME scheme. To support traceability, user's identity is encrypted via the binary-bit variant of Linder's LWE-based scheme [29] with Naor-Yung double-encryption paradigm [38], which also achieves a stronger unlinkability. Besides, an extra NIZKAoK is designed for TA to produce a proof of the tracing result. The inherent computationally simulatable property of the underlying ZKAoK also equips our scheme with full deniability.

5.1. Description of the scheme

- **Setup.** Given a security parameter λ , this algorithm specifies the following:
 - A maximum number of group members $N = 2^\ell = \text{poly}(\lambda)$.
 - Dimension $n = O(\lambda)$, a large enough prime module q , and $k = \lceil \log q \rceil$.
 - Gadget matrix $\mathbf{G} = \mathbf{I}_n \otimes (1 \ 2 \ 4 \dots 2^{k-1}) \in \mathbb{Z}_q^{n \times nk}$.
 - Matrix dimension $m = 2nk$ for the accumulator layer, and $n_1, n_2 = \text{poly}(\lambda)$ for the encryption layer.
 - Matrix dimensions $n_3 = \text{poly}(\lambda)$, $m_1 = O(n_3)$, integer module $q_1 \geq 2^{O(n_3)}$ and $\theta \geq \frac{2\lambda}{n_3 m_1}$ for KE.
 - Discrete Gaussian distribution χ over $\mathbb{Z}$ with deviation $\sigma > \sqrt{\frac{2n_3}{\pi}}$.
 - Uniformly random matrices $\mathbf{A} = [\mathbf{A}_1 | \mathbf{A}_2] \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{K} \in \mathbb{Z}_{q_1}^{n_3 \times n_3}$.
 - $F : \mathbb{Z}_{q_1}^{n_3 \times m_1} \rightarrow [-p, p]^t$ is an injective mapping, where p, t are defined in [50]. F^{-1} is the inverse of F .
 - $\mathcal{H} : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ is a collision-resistant hash function.
- Outputs global public parameters

$$\text{par} = \{N, \ell, n, q, k, m, n_1, n_2, n_3, m_1, q_1, \theta, \chi, \sigma, \mathbf{G}, \mathbf{A}, \mathbf{K}, F, F^{-1}, \mathcal{H}\}.$$

- **KGen_{TA}.** Given par , TA initializes the Naor-Yung mechanism with the binary-bit encryption scheme [29]. More precisely, for $i \in \{1, 2\}$, it first samples $\mathbf{B} \xleftarrow{\$} \mathbb{Z}_q^{n_1 \times n_2}$, $\mathbf{S}_i \xleftarrow{\$} \{0, 1\}^{\ell \times n_1}$ and $\mathbf{E}_i \xleftarrow{\$} \{0, 1\}^{\ell \times n_2}$, then computes $\mathbf{P}_i = \mathbf{S}_i \cdot \mathbf{B} + \mathbf{E}_i \in \mathbb{Z}_q^{\ell \times n_2}$. The tracing key pair is set as $\text{tsk} = \mathbf{S}_1$ and $\text{tpk} = (\mathbf{B}, \mathbf{P}_1, \mathbf{P}_2)$.
- **CreateGroup.** On input par , GA establishes a new group G as follows.
 - Samples $\text{msk} \xleftarrow{\$} \{0, 1\}^m$ and compute $\text{mpk} = \mathbf{A} \cdot \text{msk} \bmod q$, which is viewed as an identifier of G . Besides, assume there is an authentication channel, such that only GA (i.e., the entity holding msk) can edit and publish group information.
 - Initialize the following group state:
 1. Build users' registration table $\text{reg} := (\{\text{reg}[i]_{i \in [0, N-1]}\})$, where for each $i \in [0, N-1]$: $\text{reg}[i] = \mathbf{0}^{nk}$. In the long term, $\text{reg}[i]$ will be used to store the public key of a registered user.
 2. Build a Merkle tree $\mathcal{T}_G$ on top of $\{\text{reg}[i]_{i \in [0, N-1]}\}$. Note that $\mathcal{T}_G$ is an all-zero tree here, yet it will be changed when membership enrollment or revocation occurs.
 3. Build users' reservation list $\text{res} := (\{\text{res}[i]_{i \in [0, N-1]}\})$ as $\text{res}[i] = \text{bin}_\ell(i)$. It will store the locations of the zero leaves in $\mathcal{T}_G$.
 - Output $\text{gpk} = (\text{mpk}, \text{tpk})$ and distribute the initial group information $\text{info}_G = \emptyset$.
- **AddMember.** If a user U_i wants to join a group, he samples $\text{usk} := \mathbf{x}_i \xleftarrow{\$} \{0, 1\}^m$ and computes $\text{upk} := \mathbf{p}_i = \text{bin}_{nk}(\mathbf{A} \cdot \mathbf{x}_i \bmod q)$, such that $\mathbf{G} \cdot \mathbf{p}_i = \mathbf{A} \cdot \mathbf{x}_i$. W.L.O.G, assume that upk is non-zero.⁵ Next, U_i sends upk to GA who then proceeds with the procedures below.
 - Upon receiving the user's request at epoch τ_i , first verify his real identity and abort if it fails. Then issue a group identity for U_i as $\text{ID}_i = \text{res}[0] \in [0, 1]^\ell$. After that, perform the following updates:
 1. Update $\mathcal{T}_G$ by running algorithm $\text{TUpdate}_A(\text{ID}_i, \text{upk})$, denote the new root as $\mathbf{u}_{\tau_i}$.
 2. Add U_i to table reg as $\text{reg}[\text{dec}_\ell(\text{res}[0])] := \text{upk}$.
 3. Pop the front element $\text{res}[0]$ out of res .

⁵ Otherwise, U_i must either pick $\mathbf{x}_i = \mathbf{0}$ or happen to find a solution to $\text{SIS}_{n, q, \sqrt{m}}$ problem – both are negligible events.

- Let $\{p_j\}_j$ denote all the non-zero leaf nodes accumulated in the new root τ_i of the updated tree. GA invokes algorithm $\text{TWitness}_A(R, p_j)$ on $R = (\text{reg}[0], \dots, \text{reg}[N-1])$ to output a witness for each p_j :

$$w_j = (\text{ID}_j \in \{0, 1\}^\ell, (\mathbf{w}_{j,\ell}, \dots, \mathbf{w}_{j,1}) \in (\{0, 1\}^{nk})^\ell).$$

Finally, GA publishes the new group information $\text{info}_{\tau_i} = (\mathbf{u}_{\tau_i}, \{w_j\}_j)$ at the current epoch.

- U_i saves his group credential as $\text{cred}_i = (\mathbf{p}_i, w_i)$, which will be updated by GA.

- **Handshake.** At epoch τ^* , suppose a member A from group G_1 with $\text{gpk}_1 = (\text{mpk}_1, \text{tpk})$, $\text{cred}_a = (\mathbf{p}_a, w_a)$, $\text{usk}_a = \mathbf{x}_a$, and another member B from group G_2 with $\text{gpk}_2 = (\text{mpk}_2, \text{tpk})$, $\text{cred}_b = (\mathbf{p}_b, w_b)$, $\text{usk}_b = \mathbf{x}_b$, want to anonymously authenticate each other. They execute a two-round handshake as follows.

1. $A \rightarrow B : (\text{PROOF}_a)$

- ① A samples an ephemeral private key $\mathbf{S}_a \leftarrow \chi(\mathbb{Z}_{q_1}^{n_3 \times m_1})$ and a noise $\mathbf{E}_a \leftarrow \chi(\mathbb{Z}_{q_1}^{n_3 \times m_1})$. Then A computes $\mathbf{C}_a = \mathbf{K} \cdot \mathbf{S}_a + \mathbf{E}_a \in \mathbb{Z}_{q_1}^{n_3 \times m_1}$.

- ② A downloads $\mathbf{u}_{\tau_1^*}$ and her witness of the form $(\text{ID}_a, (\mathbf{w}_{a,1}, \dots, \mathbf{w}_{a,1}))$ from $\text{info}_{G_1, \tau^*}$ if it updates. Next, A double-encrypts ID_a using an LWE-based encryption scheme. Namely, for each $i \in \{1, 2\}$, A first samples $\mathbf{r}_i^{(a)} \xleftarrow{\$} \{0, 1\}^{n_2}$, $\mathbf{e}_{i,1}^{(a)} \xleftarrow{\$} \{0, 1\}^{n_1}$, $\mathbf{e}_{i,2}^{(a)} \xleftarrow{\$} \{0, 1\}^\ell$, and computes

$$\mathbf{c}_i^{(a)} = (\mathbf{c}_{i,1}^{(a)}, \mathbf{c}_{i,2}^{(a)}) = (\mathbf{B} \cdot \mathbf{r}_i^{(a)} + \mathbf{e}_{i,1}^{(a)}, \mathbf{P}_i \cdot \mathbf{r}_i^{(a)} + \mathbf{e}_{i,2}^{(a)} + \lfloor \frac{q}{2} \rfloor \cdot \text{ID}_a).$$

- ③ With public input $\text{pp}_a = (\text{par}, \text{gpk}_1, \mathbf{u}_{\tau_1^*}, \mathbf{c}_1^{(a)}, \mathbf{c}_2^{(a)})$, A invokes $\text{Send}_{\text{hs}}(\text{pp}_a, \xi_a, F(\mathbf{C}_a))$ where the secret tuple $\xi_a = (\mathbf{x}_a, \mathbf{p}_a, \text{ID}_a, \mathbf{w}_{a,1}, \dots, \mathbf{w}_{a,1}, \mathbf{r}_1^{(a)}, \mathbf{r}_2^{(a)}, \mathbf{e}_{1,1}^{(a)}, \mathbf{e}_{1,2}^{(a)}, \mathbf{e}_{2,1}^{(a)}, \mathbf{e}_{2,2}^{(a)})$ satisfies:

$$\begin{cases} \text{TVerify}_A(\mathbf{u}_{\tau^*}, \mathbf{p}_a, (\text{ID}_a, (\mathbf{w}_{a,1}, \dots, \mathbf{w}_{a,1}))) = 1; \\ \mathbf{A} \cdot \mathbf{x}_a = \mathbf{G} \cdot \mathbf{p}_a \pmod{q}, \mathbf{p}_a \neq \mathbf{0}^{nk}; \\ \forall i \in \{1, 2\}, \mathbf{c}_i^{(a)} = (\mathbf{c}_{i,1}^{(a)}, \mathbf{c}_{i,2}^{(a)}) = (\mathbf{B} \cdot \mathbf{r}_i^{(a)} + \mathbf{e}_{i,1}^{(a)}, \mathbf{P}_i \cdot \mathbf{r}_i^{(a)} + \mathbf{e}_{i,2}^{(a)} + \lfloor \frac{q}{2} \rfloor \cdot \text{ID}_a). \end{cases} \quad (5)$$

Note that if unfold the above Set (5) in the form of linear equations, they are exactly Set (2) handled in Section 4.4.

- ④ A sends $\text{PROOF}_a = (\Sigma_a, \mathbf{c}_1^{(a)}, \mathbf{c}_2^{(a)})$ to B .

Remark 3. We stress that in the above step ③, algorithm Send_{hs} is of the AME scheme designed in Section 4.4, which follows construction II proposed in Section 4.3. Besides, recall that we omit algorithm Setup_{hs} in Section 4.4, so we argue that there is an implicit Setup_{hs} within our SH scheme. In the context of our AME scheme, the “credential” of A is her secret witness ξ_a , of which some elements are A ’s secret key and credential in an SH group, and others are the randomness used in her identity’s encryption. From this view, we can conceptually regard algorithms CreateGroup and AddMember of our SH scheme as a dynamic-setting instantiation for Setup_{hs} of the underlying AME scheme, namely, GA plays the role of a trusted authority in Definition 10.

2. $B \rightarrow A : (\text{PROOF}_b, \mathbf{V}_b)$

- ① B samples his ephemeral private key $\mathbf{S}_b \leftarrow \chi(\mathbb{Z}_{q_1}^{m_1 \times n_3})$ and a noise $\mathbf{E}_b \leftarrow \chi(\mathbb{Z}_{q_1}^{m_1 \times n_3})$. Then B computes $\mathbf{C}_b = \mathbf{K} \cdot \mathbf{S}_b + \mathbf{E}_b \in \mathbb{Z}_{q_1}^{m_1 \times n_3}$.

- ② B also downloads $\mathbf{u}_{\tau_2^*}$ and his witness w_b from $\text{info}_{G_2, \tau^*}$ if it updates. Next, through using the same encryption scheme, B double-encrypts his identity ID_b as $(\mathbf{c}_1^{(b)}, \mathbf{c}_2^{(b)})$.

- ③ With public input $\text{pp}_b = (\text{par}, \text{gpk}_2, \mathbf{u}_{\tau_2^*}, \mathbf{c}_1^{(b)}, \mathbf{c}_2^{(b)})$, B utilizes algorithm $\text{Send}_{\text{hs}}(\text{pp}_b, \xi_b, F(\mathbf{C}_b^\top))$ to produce a text Σ_b for his secret tuple $\xi_b = (\mathbf{x}_b, \mathbf{p}_b, \text{ID}_b, \mathbf{w}_{b,1}, \dots, \mathbf{w}_{b,1}, \mathbf{r}_1^{(b)}, \mathbf{r}_2^{(b)}, \mathbf{e}_{1,1}^{(b)}, \mathbf{e}_{1,2}^{(b)}, \mathbf{e}_{2,1}^{(b)}, \mathbf{e}_{2,2}^{(b)})$, which satisfies the similar constraints as Set (5).

- ④ With $\text{pp}'_a = (\text{par}, \text{gpk}_2, \mathbf{u}_{\tau_2^*}, \mathbf{c}_1^{(a)}, \mathbf{c}_2^{(a)})$, B runs $\text{Open}_{\text{hs}}(\text{pp}'_a, \Sigma_a)$ (recall that algorithm Open_{hs} is also of the AME scheme designed in Section 4.4), to retrieve the hidden message M_a , so that he can recover the matrix of A as $\mathbf{C}'_a = F^{-1}(M_a)$.

- ⑤ B samples another noise $\tilde{\mathbf{E}}_b \xleftarrow{\$} \chi(\mathbb{Z}_{q_1}^{m_1 \times m_1})$ and sets $\mathbf{V}_b = \mathbf{S}_b \cdot \mathbf{C}'_a + \tilde{\mathbf{E}}_b$. Then B generates a check matrix $\mathbf{M} \in \mathbb{Z}_2^{m_1 \times m_1}$ holding that $\mathbf{M}[i, j] = \text{floor}(\frac{2^{\theta+1}}{q_1} \cdot \mathbf{V}_b[i, j]) \pmod{2}$. Hereafter we view each entry of $\mathbf{V}_b$ as an integer in $[\frac{-q_1}{2}, \frac{q_1}{2} - 1]$.

- ⑥ B generates the shared session key $\mathbf{K}_b \in \mathbb{Z}_{2^\theta}^{m_1 \times m_1}$ by rounding the θ most significant bits from each entry of $\mathbf{V}_b$, i.e., $\mathbf{K}_b[i, j] = \left\lfloor \frac{2^\theta \cdot \mathbf{V}_b[i, j]}{q_1} \right\rfloor \pmod{2^\theta}$. Then he computes $\mathbf{V}_b = \mathcal{H}(\mathbf{K}_b, \mathbf{C}_b, 0)$.

- ⑦ B sets $\text{PROOF}_b = (\Sigma_b, \mathbf{c}_1^{(b)}, \mathbf{c}_2^{(b)}, \mathbf{M})$ and sends $(\text{PROOF}_b, \mathbf{V}_b)$ to A .

3. $A \rightarrow B : (\mathbf{V}_a)$

- ① A invokes $\text{Open}_{\text{hs}}(\text{pp}'_b, \Sigma_b)$, where $\text{pp}'_b = (\text{par}, \text{gpk}_1, \mathbf{u}_{\tau_1^*}, \mathbf{c}_1^{(b)}, \mathbf{c}_2^{(b)})$, to get M_b , and recovers the transported matrix $\mathbf{C}'_b = F^{-1}(M_b)$.

- ② A computes a value $\mathbf{V}_a = \mathbf{C}'_b \cdot \mathbf{S}_a$. Next, she extracts the shared session key $\mathbf{K}_a \in \mathbb{Z}_{2^\theta}^{m_1 \times m_1}$ from $\mathbf{V}_a$ via a reconciliation technique, i.e., $\mathbf{K}_a[i, j] = \left\lfloor \frac{2^\theta \cdot \mathbf{V}_a[i, j]}{q_1} + \frac{2\mathbf{M}[i, j]-1}{4} \right\rfloor \bmod 2^\theta$.
- ③ A verifies the response $\mathbf{V}_b$ by equation $\mathbf{V}_b \stackrel{?}{=} \mathcal{H}(\mathbf{K}_a, \mathbf{C}'_b, 0)$. If so, A outputs 1 and sends $\mathbf{V}_a = \mathcal{H}(\mathbf{K}_a, \mathbf{C}_a, 1)$ to B . Else, A outputs 0 and replies a random value $\mathbf{V}_a \xleftarrow{\$} \mathbb{Z}_q$.
- ④ B verifies $\mathbf{V}_a$ via a similar equation $\mathbf{V}_a \stackrel{?}{=} \mathcal{H}(\mathbf{K}_b, \mathbf{C}'_a, 1)$. He outputs 1 if the equation holds and 0 otherwise.

Remark 4. Note that the above steps ①–④ clarify how users can utilize the AME scheme designed in Section 4.4, together with the KE protocol [8], to produce a checking code $(\mathbf{V}_a/\mathbf{V}_b)$ for mutual authentication and negotiate a session key $(\mathbf{K}_a/\mathbf{K}_b)$ for subsequent communication. Further, this strategy can work as a generic approach from AME plus KE-like protocol, to Handshake of an SH.

- **TraceMember.** When disputes happen at epoch τ^* , TA first retrieves the transcript $\text{PROOF} = (\Sigma_j, \mathbf{c}_1, \mathbf{c}_2)$. Then to publish the tracing result and its proof $(\text{ID}, \pi_{\text{trace}}, \mathbf{c}_1)$, TA performs the following steps.
 - Decrypt $\mathbf{c}_1 = (\mathbf{c}_{1,1}, \mathbf{c}_{1,2})$ to get the user's identity $\text{ID} : \mathbf{j} = (j_1, \dots, j_\ell) \in \{0, 1\}^\ell$. Namely, first compute $\mathbf{m} = \mathbf{c}_{1,2} - \mathbf{S}_1 \cdot \mathbf{c}_{1,1} \in \{0, 1\}^\ell$, then for $i \in [1, \ell]$, set $j_i = 0$ if $\|\mathbf{m}_i\| < \lfloor q/4 \rfloor$ and 1 otherwise.
 - If ID does not match any witness included in info_{τ^*} , return $\perp$.
 - If the registration record $\text{reg}[\text{dec}_\ell(\text{ID})]$ in table reg is $\mathbf{0}^{nk}$, return $\perp$.
 - With public input $(\mathbf{j}, \text{gpk}, \mathbf{c}_1, \mathbf{c}_2)$, apply system Π_{trace} (details are presented in Section 5.2) to generate an NIZKAoK π_{trace} , demonstrating the possession of $\mathbf{S}_1 \in \{0, 1\}^{\ell \times n_1}$, $\mathbf{E}_1 \in \{0, 1\}^{\ell \times n_2}$ and $\mathbf{t} \in [-n_1, n_2 + 1]^\ell$, which satisfy that:

$$\begin{cases} \mathbf{S}_1 \cdot \mathbf{B} + \mathbf{E}_1 = \mathbf{P}_1, \\ \mathbf{S}_1 \cdot \mathbf{c}_{1,1} + \mathbf{t} = \mathbf{c}_{1,2} - \lfloor \frac{q}{2} \rfloor \cdot \mathbf{j}. \end{cases} \quad (6)$$

- **JudgeMember.** By regarding the public data $(\text{gpk}, \text{ID}, \pi_{\text{trace}}, \mathbf{c}_1)$ as the input of Verify (recalled in Section 2.4), this algorithm checks if π_{trace} is valid. It returns 1 if it is the case, otherwise returns 1.
- **RemoveMember.** To revoke an active user U_r at epoch τ_r , GA works as follows to update the group.
 - To remove a member U_r with identity ID_r from group G_r , apply $\text{TUpdate}_A(\text{ID}_r, \mathbf{0}^{nk})$ to update $\mathcal{T}_{G_r}$.
 - Reset $\text{reg}[\text{dec}_\ell(\text{ID}_r)] := \mathbf{0}^{nk}$, then push ID_r into res to record the location of the new zero leaf.
 - Publish updated information in the same manner of AddMember , i.e., output $\text{info}_{\tau_r} = (\mathbf{u}_{\tau_r}, \{w_j\}_j)$.

FD-completeness. We demonstrate that our scheme is *FD-completeness* with overwhelming probability if A and B belong to the same group, and both follow the specified protocol.

First, by *correctness* of utilized AME, both A and B can retrieve the correct hidden message $\mathbf{C}' = \mathbf{C}$. In that sense, we know that $\mathbf{K}_a \neq \mathbf{K}_b$ with negligible probability from [8]. Therefore, both members can verify the corresponding equations successfully, i.e., the message authentication code $\mathbf{V}_a$ ($\mathbf{V}_b$) is valid, and consequently, the handshake protocol will output 1.

Next, we show that algorithm TraceMember will output ID_a (ID_b) and produce a valid proof π_{trace} with overwhelming probability. Observe that the decryption procedure essentially computes

$$\mathbf{m} = \mathbf{c}_{1,2} - \mathbf{S}_1 \cdot \mathbf{c}_{1,1} = \mathbf{t} + \lfloor \frac{q}{2} \rfloor \cdot \text{ID}_a \bmod q,$$

where $\mathbf{t} = \mathbf{E}_1 \cdot \mathbf{r}_1 + \mathbf{e}_{1,2} - \mathbf{S}_1 \cdot \mathbf{e}_{1,1}$, then sets i -th bit of $\mathbf{j}$ to be 0 if i -th bit of $\mathbf{m}$ is less than $\lfloor \frac{q}{4} \rfloor$ and 1 otherwise. By setting appropriate parameters (see next subsection), we always have $\|\mathbf{t}\|_\infty \leq \max(n_1, n_2 + 1) \ll \frac{q}{4}$, inducing that $\mathbf{j} = \text{ID}_a$ with overwhelming probability. Further, on condition that A is active, there must exist a non-zero leaf in Merkle-tree $\mathcal{T}_a$ and the corresponding $\text{reg}[\text{dec}_\ell(\mathbf{j})]$ cannot be $\mathbf{0}^{nk}$. Thus, TraceMember will go to step 4, where it can obtain the tuple $(\mathbf{S}_1, \mathbf{E}_1, \mathbf{t})$ satisfying Set (6). Finally, by *completeness* of system Π_{trace} , π_{trace} will always be accepted.

5.2. NIZKAoK for tracing authority

Tracing authority of our scheme needs to additionally output an argument π_{trace} , showing that it possesses tsk and conducts the tracing algorithm correctly. The structure of system Π_{trace} is as follows.

Public input: $\mathbf{B}, \mathbf{P}, \mathbf{c}_1, \mathbf{c}_2, \mathbf{j}$.

Prover's witness: $\mathbf{S} \in \{0, 1\}^{\ell \times n_1}, \mathbf{E} \in \{0, 1\}^{\ell \times n_2}, \mathbf{t} \in [-n_1, n_2 + 1]^\ell$.

Prover's goal: Convince the verifier in zero-knowledge that Set (6) holds.

We also build the system by reducing Set (6) to a union relation

$$\mathcal{R}_{\text{trace}} = \{(\mathbf{M}_4, \mathbf{y}_4, \mathcal{L}_4), (\mathbf{x}_4) : \mathbf{M}_4 \cdot \mathbf{x}_4 = \mathbf{y}_4 \wedge \forall (h, i, j) \in \mathcal{L}_4, \mathbf{x}_4[h] = \mathbf{x}_4[i] \cdot \mathbf{x}_4[j]\}.$$

Towards this goal, conduct the following steps:

1. Let $\mathbf{n}_1 = (n_1 \dots n_1)^\ell$ and set $\mathbf{t}' = \mathbf{t} + \mathbf{n}_1 \in [0, n_3]^\ell$ where $n_3 = n_1 + n_2 + 1$. Then decompose $\mathbf{t}'$ into vector $\hat{\mathbf{t}}$, holding that $\hat{\mathbf{G}} \cdot \hat{\mathbf{t}} = \mathbf{t}'$, where the gadget matrix $\hat{\mathbf{G}} = \mathbf{I}_\ell \otimes \mathbf{g}_{\delta_{n_3}}$.
2. Denote $\mathbf{S}^\top = (\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_\ell)$, $\mathbf{E}^\top = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell)$, $\mathbf{P}^\top = (\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_\ell)$, and $\mathbf{c}_2^\top - \lfloor q/2 \rfloor \cdot \mathbf{j}'^\top = (j'_1 \ j'_2 \dots j'_\ell)$. Define $\hat{\mathbf{B}}_i = (\mathbf{0}^i, \mathbf{B}, \mathbf{0}^{\ell-1}, \mathbf{I}_\ell, \mathbf{0}^{\ell-i-1})$, $\tilde{\mathbf{C}}_i = (\mathbf{0}^i, \mathbf{c}_1, \mathbf{0}^{2\ell-i-1})$ for all $i \in [0, \ell - 1]$.
3. Set $\mathbf{x}_4 = (\mathbf{s}_1^\top \ \mathbf{s}_2^\top \ \dots \ \mathbf{s}_\ell^\top \ \mathbf{e}_1^\top \ \mathbf{e}_2^\top \ \dots \ \mathbf{e}_\ell^\top \ \hat{\mathbf{t}}^\top)^\top$, and public inputs

$$\mathbf{M}_4 = \begin{pmatrix} \hat{\mathbf{B}}_1 & \dots & \hat{\mathbf{B}}_\ell & \tilde{\mathbf{C}}_1 & \dots & \tilde{\mathbf{C}}_\ell \\ \mathbf{0} & \dots & \mathbf{0} & \hat{\mathbf{G}} & \dots & \hat{\mathbf{G}} \end{pmatrix}^\top, \mathbf{y}_4 = (\mathbf{p}_1 \ \dots \ \mathbf{p}_\ell \ j'_1 \ \dots \ j'_\ell)^\top.$$

The quadratic constraints over $\mathbf{x}_4$ are $\mathcal{L}_4 = \{i, i, i\}$, $i \in [1, (n_1 + n_2 + \lceil \log(n_3 + 1) \rceil)\ell]$.

5.3. Security analysis

Below we prove that our FDSH meets all the desired properties in the random oracle model, under the SIS, LWE and BLWE assumptions.

Theorem 3. Assume the hardness of $\text{SIS}_{n,q,\sqrt{m}}$ problem, our scheme provides FD-impersonator resistance.

Proof. We prove this property by contradiction. Suppose that $\mathcal{A}$ succeeds in $\text{Exp}_{\mathcal{A}}^{\text{IR}}$ with non-negligible advantage ϵ . Then we can build a PPT algorithm $\mathcal{B}$ that either breaks the security of updatable accumulator or solves $\text{SIS}_{n,q,\sqrt{m}}$ problem (both are related to matrix $\mathbf{A}$), with also non-negligible probability.

Given $\mathbf{A}$ for either situation that $\mathcal{B}$ is in, it first generates par as that in Setup. Then $\mathcal{B}$ proceeds as described in $\text{Exp}_{\mathcal{A}}^{\text{IR}}$, where it can adaptively answer all the oracle queries made by $\mathcal{A}$. After $\mathcal{A}$ wins with its interactive output PROOF^* of the handshake protocol, $\mathcal{B}$ performs the following.

Parse the transcript PROOF^* as $(\Sigma^*, \mathbf{c}_1^*, \mathbf{c}_2^*)$. By soundness of used AME, Σ^* must be a valid text generated from Send_{hs} . Thus, we can argue that $\mathcal{A}$ must have queried the related random oracle $\mathcal{G}$ via Fiat-Shamir heuristic on input $\eta^* = (\widehat{\text{cm}}_{\eta^*}^*, \overline{\text{cm}}_{\eta^*}^*, \text{pp}^*)$. Otherwise, guessing correctly this value occurs only with negligible probability $(\frac{1}{2^{p+1}})^\ell$. Therefore, with probability at least $\epsilon - (\frac{1}{2^{p+1}})^\ell$, the tuple η^* has been an input of one hash query, denoted as $\kappa^* \leq q_{\mathcal{G}}$, where $q_{\mathcal{G}}$ is the total number of queries to $\mathcal{G}$.

Next, $\mathcal{B}$ picks κ^* as the target forking point and reruns $\mathcal{A}$ polynomial times with the same random input as in the original execution. But from the κ^* -th query onwards, $\mathcal{B}$ replies to $\mathcal{A}$ with fresh and independent hash values. Note that the input of κ^* hash query must be η^* , then the Forking Lemma in [11] implies that, with probability larger than $\frac{1}{2}$, $\mathcal{B}$ can obtain 3 forks involving the same tuple η^* , but with pairwise distinct challenges $\widehat{ch}_{\eta^*,1}, \widehat{ch}_{\eta^*,2}, \widehat{ch}_{\eta^*,3} \in [-p, p]^\ell$. Moreover, by the binding property of used commitment scheme, $\mathcal{B}$ can obtain 3 valid tuples from the output of $\mathcal{A}$ as

$$\{(ch_{\eta^*,1}, \{\widehat{\text{cm}}_{\eta^*}^*, \overline{\text{cm}}_{\eta^*}^*\}, \text{rsp}_{\eta^*,1}), (ch_{\eta^*,2}, \{\widehat{\text{cm}}_{\eta^*}^*, \overline{\text{cm}}_{\eta^*}^*\}, \text{rsp}_{\eta^*,2}), (ch_{\eta^*,3}, \{\widehat{\text{cm}}_{\eta^*}^*, \overline{\text{cm}}_{\eta^*}^*\}, \text{rsp}_{\eta^*,3})\},$$

by first recovering the unsent $\overline{\text{cm}}_{\eta^*}^*$ and then the original ch_{η^*} . Next, by proof of knowledge of Π_{hs} , $\mathcal{B}$ can extract the witness $\xi' = (\mathbf{x}', \mathbf{p}', \text{ID}', \mathbf{w}'_{1,\tau}, \dots, \mathbf{w}'_{l,\tau}, \mathbf{r}', \mathbf{r}'_2, \mathbf{e}'_{1,1}, \mathbf{e}'_{1,2}, \mathbf{e}'_{2,1}, \mathbf{e}'_{2,2})$, which holds that

$$\begin{cases} \text{TVerify}_{\mathbf{A}}(\mathbf{u}_\tau, \mathbf{p}', (\text{ID}', (\mathbf{w}'_{1,\tau}, \dots, \mathbf{w}'_{l,\tau}))) = 1; \\ \mathbf{A} \cdot \mathbf{x}' = \mathbf{G} \cdot \mathbf{p}' \pmod{q}, \mathbf{p}' \neq \mathbf{0}^{nk}; \\ \forall i \in [1, 2], \mathbf{c}_i^* = (\mathbf{c}'_{i,1}, \mathbf{c}'_{i,2}) = (\mathbf{B} \cdot \mathbf{r}'_i + \mathbf{e}'_{i,1}, \mathbf{P}_i \cdot \mathbf{r}'_i + \mathbf{e}'_{i,2} + \lfloor \frac{q}{2} \rfloor \cdot \text{ID}'). \end{cases} \quad (7)$$

Finally, consider the following two cases:

Case 1: ID' has been revoked at current epoch τ , i.e., the value of leaf ID' is $\mathbf{0}^{nk}$. This violates the security of the utilized accumulator since we have $\text{TVerify}_{\mathbf{A}}(\mathbf{u}_\tau, \mathbf{p}', (\text{ID}', \mathbf{w}')) = 1$ with $\mathbf{p}' \neq \mathbf{0}$ at leaf ID' .

Case 2: ID' is active at current epoch τ . The condition that $G' \notin \text{CG}$ ensures that ID' is not a dummy user created by $\mathcal{A}$. Moreover, the fact that $(\text{ID}', \cdot) \notin \text{CU}$ indicates $\mathcal{A}$ does not know the secret key $\text{usk}_{j^*} = \mathbf{x}_{j^*}$ of ID' , where $\mathbf{x}_{j^*}$ was randomly chosen by $\mathcal{B}$ holding that $\mathbf{A} \cdot \mathbf{x}_{j^*} = \mathbf{G} \cdot \mathbf{p}'$. Then we claim that $\mathbf{x}_{j^*} \neq \mathbf{x}'$ with high probability. Therefore, we find a small vector $\mathbf{a} = \mathbf{x}_{j^*} - \mathbf{x}'$ satisfying $\mathbf{A} \cdot \mathbf{a} = \mathbf{0} \pmod{q}$. Recall that $\mathbf{x}_{j^*}, \mathbf{x}' \in \{0, 1\}^m$, which induces $\|\mathbf{a}\| \leq \sqrt{m}$. Thus $\mathcal{B}$ solves an $\text{SIS}_{n,q,\sqrt{m}}$ problem with non-negligible probability. This concludes the proof. $\square$

Theorem 4. Assume the hardness $\text{BLWE}_{n_1,q}$ and $\text{BLWE}_{n_2,q}$ problems, our scheme meets FD-detector resistance.

Proof. Let $\mathcal{A}, \mathcal{B}$ be PPT algorithms, playing the role of adversary and challenger in $\text{Exp}_{\mathcal{A}}^{\text{DR-b}}$ respectively. We define a sequence of hybrid games, where the first game is $\text{Exp}_{\mathcal{A}}^{\text{DR-0}}$ and the last one is $\text{Exp}_{\mathcal{A}}^{\text{DR-1}}$, then prove that any two consecutive

games are indistinguishable. For each i , denote by R_i the output of the adversary in game i . The concrete games are as follows.

Game 0: This is exactly the original game $\text{Exp}_{\mathcal{A}}^{\text{DR}-0}$.

Game 1: This game is the same as Game 0 with only one modification: add (S_2, E_2) to tsk . This makes no difference from the view of $\mathcal{A}$. Hence we have $\Pr[R_1 = 1] = \Pr[\text{Exp}_{\mathcal{A}}^{\text{DR}-0} = 1]$.

Game 2: This game is identical to Game 1 except that it generates a simulated proof for oracle Trace via running the simulator of system Π_{trace} even though $\mathcal{B}$ possesses correct witnesses for producing a real proof. The view of $\mathcal{A}$, however, is computationally indistinguishable between Game 1 and Game 2 by *honest-verifier zero-knowledge* of system Π_{trace} . So $\Pr[R_1 = 1] - \Pr[R_2 = 1] = \text{negl}(\lambda)$.

Game 3: This game uses S_2 instead of S_1 to simulate the oracle Trace. The view of $\mathcal{A}$ is also the same as in Game 2 until event F_1 happens, i.e., $\mathcal{A}$ queries the Trace oracle its handshake transcript $(\Pi_{\mathcal{A}}, c_1, c_2, \text{info}_\tau)$ with c_1, c_2 encrypting distinct identity strings. Since F_1 breaks *soundness* of used AME, we have $|\Pr[R_2 = 1] - \Pr[R_3 = 1]| \leq \Pr[F_1] \leq \text{Adv}_{\text{AME}}^{\text{sound}}(\lambda) = s$.

Game 4: This game generates a simulated text when $\mathcal{B}$ executes a handshake with $\mathcal{A}$, which is computationally indistinguishable to $\mathcal{A}$ by *full anonymity* of used AME, thus $\Pr[R_3 = 1] - \Pr[R_4 = 1] = \text{negl}(\lambda)$.

Game 5: This is the same as Game 4 except that c_1 is now encryption of ID_r while c_2^* keeps encryption of ID^* for query $\text{Chal}_b^{\text{DR}}$. By the semantic security of our underlying encryption scheme (which relies on BLWE assumption), this modification is negligible to $\mathcal{A}$ when executing a handshake with $\mathcal{B}$. In addition, recall that we use S_2 for oracle Trace, this change also makes no difference to the view of $\mathcal{A}$ querying Trace. Combine the discussion above, we have $\Pr[R_4 = 1] - \Pr[R_5 = 1] = \text{negl}(\lambda)$.

Game 6: This game follows Game 5 with one change: it switches back to use S_1 for Trace and deserts (S_2, E_2) . This modification is indistinguishable to $\mathcal{A}$ until events F_2 , where $\mathcal{A}$ queries Trace with its transcript $(\Pi_{\mathcal{A}}, c_1, c_2, \text{info}_\tau)$ where c_1, c_2 encrypt different identities, occurs. However, event F_2 also contradicts *soundness* of used AME. Therefore, we have $\Pr[R_5 = 1] - \Pr[R_6 = 1] \leq \text{Adv}_{\text{AME}}^{\text{sound}}(\lambda) = s$.

Game 7: In this game, we modify again the output of $\mathcal{B}$ at the challenge step. Namely, c_2 is also computed by encrypting ID_r . By the semantic security of our encryption scheme for public key (B, P) , this change is negligible to $\mathcal{A}$. Note that we now use S_1 for the oracle Trace. Thus changing c_2 makes no difference to the view of $\mathcal{A}$. Hence it satisfies that $\Pr[R_6 = 1] - \Pr[R_7 = 1] = \text{negl}(\lambda)$.

Game 8: Now we generate a real text when $\mathcal{B}$ simulates a handshake protocol at the challenge step. Note that at this point $\mathcal{B}$ actually acts as ID_r . By *full anonymity* of utilized AME, the view of $\mathcal{A}$ in Game 7 and Game 8 is computationally indistinguishable, i.e., we have $\Pr[R_7 = 1] - \Pr[R_8 = 1] = \text{negl}(\lambda)$.

Game 9: This game produces a real proof for Trace. By *honest-verifier zero-knowledge* of Π_{trace} , we have Game 8 and Game 9 are computationally indistinguishable to $\mathcal{A}$, which means $\Pr[R_8 = 1] - \Pr[R_9 = 1] = \text{negl}(\lambda)$. This game is exactly the experiment $\text{Exp}_{\mathcal{A}}^{\text{DR}-1}$. Hence, we have $\Pr[R_9 = 1] = \Pr[\text{Exp}_{\mathcal{A}}^{\text{DR}-1} = 1]$.

Tracing through the whole games, it is clear that $|\Pr[\text{Exp}_{\mathcal{A}}^{\text{DR}-1} = 1] - \Pr[\text{Exp}_{\mathcal{A}}^{\text{DR}-0} = 1]| = \text{negl}(\lambda)$, so that *FD-detector resistance* is proved. $\square$

Theorem 5. Assume the hardness of $\text{BLWE}_{n_1, q}$ and $\text{BLWE}_{n_2, q}$ problems, our scheme provides *FD-unlinkability*.

Proof. The proof of Theorem 5 is similar to that of Theorem 4. Based on the security of the underlying encryption scheme, our lattice-based AME and system Π_{trace} , we can also build a sequence of indistinguishable games to demonstrate that an adversary can gain no extra advantage except for randomly guessing the target bit b . Below we use the same notations as the proof of Theorem 5. The formal descriptions of these games are as follows.

Game 0: This is exactly the original game $\text{Exp}_{\mathcal{A}}^{\text{Unlink}-0}$, where $\mathcal{B}$ simulates the same user ID_0 to perform two handshakes with $\mathcal{A}$.

Game 1: This game is the same as Game 0 with only one change: add (S_2, E_2) to tsk . This brings no difference from the view of $\mathcal{A}$. Hence we have $\Pr[R_1 = 1] = \Pr[\text{Exp}_{\mathcal{A}}^{\text{Unlink}-0} = 1]$.

Game 2: This game is identical to Game 1 except that it generates a simulated proof for oracle Trace via running the simulator of system Π_{trace} . While the view of $\mathcal{A}$ is computationally indistinguishable between Game 1 and Game 2 by *honest-verifier zero-knowledge* of system Π_{trace} . So $\Pr[R_1 = 1] - \Pr[R_2 = 1] = \text{negl}(\lambda)$.

Game 3: This game utilizes S_2 instead of S_1 in response to queries for oracle Trace. The view of $\mathcal{A}$ is also the same as in Game 2 until event F_1 happens, i.e., $\mathcal{A}$ queries Trace the handshake transcript $(\Pi_{\mathcal{A}}, c_1, c_2, \text{info}_\tau)$ with c_1, c_2 encrypting distinct identity strings. However, F_1 breaks *soundness* of our AME scheme, thus we have $|\Pr[R_2 = 1] - \Pr[R_3 = 1]| \leq \Pr[F_1] \leq \text{Adv}_{\text{AME}}^{\text{sound}}(\lambda) = s$.

Game 4: This game proceeds the same as Game 3 when $\mathcal{B}$ handshakes with $\mathcal{A}$ for the first time. Yet it produces a simulated text when executing the second handshake with $\mathcal{A}$ (by acting as ID_0). Note that here $\mathcal{A}$ can obtain the group credentials of ID_0 . However, we argue that even under this powerful exposure case for $\mathcal{A}$, this change is still

computationally indistinguishable from the view of $\mathcal{A}$, thanks to *full anonymity* of our well-designed AME,⁶ thus it holds $\Pr[R_3 = 1] - \Pr[R_4 = 1] = \text{negl}(\lambda)$.

Game 5: For the second handshake with $\mathcal{A}$, this game modifies $\mathbf{c}_1$ to be encryption of ID_1 while $\mathbf{c}_2^*$ keeps encryption of ID_0 . By the semantic security of the underlying encryption scheme, this change is negligible to $\mathcal{A}$. In addition, recall that we use $\mathbf{S}_2$ for oracle Trace, this change also makes no difference when $\mathcal{A}$ queries Trace. Based on the analysis above, we have $\Pr[R_4 = 1] - \Pr[R_5 = 1] = \text{negl}(\lambda)$.

Game 6: This game switches back to use $\mathbf{S}_1$ and discards $(\mathbf{S}_2, \mathbf{E}_2)$. This modification is indistinguishable to $\mathcal{A}$ until events F_2 , where $\mathcal{A}$ queries Trace with transcript $(\Pi_{\mathcal{A}}, \mathbf{c}_1, \mathbf{c}_2, \text{info}_\tau)$ where $\mathbf{c}_1, \mathbf{c}_2$ encrypt different identities, occurs. However, event F_2 also contradicts *soundness* of used AME scheme. Hence we have $\Pr[R_5 = 1] - \Pr[R_6 = 1] \leq \text{Adv}_{\text{AME}}^{\text{sound}}(\lambda) = s$.

Game 7: In this game, we modify again the output of $\mathcal{B}$ for the second handshake with $\mathcal{A}$. Namely, $\mathbf{c}_2$ is the encryption ID_1 . This change is also negligible to $\mathcal{A}$ due to the semantic security of the underlying encryption scheme. Besides, since we now use $\mathbf{S}_1$ for Trace, changing $\mathbf{c}_2$ makes no difference to the view of $\mathcal{A}$ for querying Trace. Thus it holds $\Pr[R_6 = 1] - \Pr[R_7 = 1] = \text{negl}(\lambda)$.

Game 8: Now for the second handshake, $\mathcal{B}$ turns to honestly simulate ID_1 to interact with $\mathcal{A}$. Due to the same reason we expound in Game 4, the view of $\mathcal{A}$ in Game 7 and Game 8 is computationally indistinguishable by *full anonymity* of the well-designed AME, even if $\mathcal{A}$ can get access to the group credential of ID_1 . Thus we have $\Pr[R_7 = 1] - \Pr[R_8 = 1] = \text{negl}(\lambda)$.

Game 9: Finally, this game outputs a real proof for Trace. By *honest-verifier zero-knowledge* of Π_{trace} , we have Game 8 and Game 9 are computationally indistinguishable to $\mathcal{A}$, which means $\Pr[R_8 = 1] - \Pr[R_9 = 1] = \text{negl}(\lambda)$. In this way, we move to the experiment $\text{Exp}_{\mathcal{A}}^{\text{Unlink}-1}$, where $\mathcal{B}$ successionaly simulates ID_0 and ID_1 to handshake with $\mathcal{A}$, so we have $\Pr[R_9 = 1] = \Pr[\text{Exp}_{\mathcal{A}}^{\text{Unlink}-1} = 1]$.

Following the above game-based discussions, it is clear that $|\Pr[\text{Exp}_{\mathcal{A}}^{\text{Unlink}-1} = 1] - \Pr[\text{Exp}_{\mathcal{A}}^{\text{Unlink}-0} = 1]| = \text{negl}(\lambda)$, so that *FD-unlinkability* is proved. $\square$

Theorem 6. Assume the hardness of $\text{SIS}_{n,q,\sqrt{m}}$ problem, our scheme provides *FD-traceability*.

Proof. Recall that $\mathcal{A}$ wins this game if Handshake outputs 1 that either tracing result is an uncorrupted user or TA outputs $\perp$ as a tracing failure. We prove that both cases occur with negligible probability.

Parse $\mathcal{A}$'s output PROOF as $(\Sigma, \mathbf{c}_1, \mathbf{c}_2)$. In this sense, we are in the same situation as that of Theorem 3. Through the same technique, we can extract a witness $\xi = (\mathbf{x}, \mathbf{p}, w_\tau, \mathbf{r}_1, \mathbf{r}_2, \mathbf{e}_{1,1}, \mathbf{e}_{1,2}, \mathbf{e}_{2,1}, \mathbf{e}_{2,2})$, where $w_\tau = (\text{ID}', \mathbf{w}_{\ell,\tau}, \dots, \mathbf{w}_{1,\tau}) \in \{0, 1\}^\ell \times (\{0, 1\}^{nk})^\ell$ holding that $\text{TVerify}_{\mathcal{A}}(\mathbf{u}_\tau, \mathbf{p}, w_\tau) = 1$, $\mathbf{p} \neq \mathbf{0}^{nk}$ and $\mathbf{c}_1, \mathbf{c}_2$ are encryptions of ID' . Then correct decryption on $\mathbf{c}_1$ in TraceMember operated by the challenger implies that $\text{ID}' = \text{ID}$ with overwhelming probability.

Now, we note that the first case means that $\mathcal{A}$ has successfully impersonated an honest user without querying the corresponding secret key, which can be used to infer a small solution $\mathbf{a}$ of $\Lambda^\perp(\mathbf{A})$ like that in Theorem 3. As for the second case, we consider two sub-cases: 1) the traced user is a revoked one, which violates the security of the underlying accumulator, because only leaf values of active users satisfy that $\text{TVerify}_{\mathcal{A}}(\mathbf{u}_\tau, \mathbf{p}, w_\tau) = 1$ and $\mathbf{p} \neq \mathbf{0}^{nk}$; 2) TA cannot generate a valid proof π_{trace} . Since the challenger possesses valid witnesses for the tracing result, this also happens with negligible probability deduced from *completeness* of system Π_{trace} . It then follows that our scheme achieves *FD-traceability*. $\square$

Theorem 7. Our scheme provides full deniability in the sense of Definition 9.

Proof. We prove it by constructing a required PPT simulator S . For chosen group G^* and epoch τ^* , the goal of S is to simulate the transcript $(\text{Proof}^*, \mathbf{v}^*)$ of some active and honest user in G^* without any group credential or user's secret key. The main supporting fact is that the underlying NIZKAoK system is (*honest-verifier*) *zero-knowledge*, i.e., has a well-designed simulator to simulate the view of an honest verifier. Here we only consider the case that $\mathcal{A}$ does not corrupt any group users, i.e., it cannot conduct a successful handshake and obtain the produced session key. The reason is that under the corruption case, $\mathcal{A}$ can unilaterally generate all the transcripts of a handshake by simply doing a handshake with herself. In this sense, it is obvious that no judge will accept $\mathcal{A}$'s proof of an accusation. Specifically, by use of a secure hash function, S performs the following steps to generate the desired elements:

- Sample $\mathbf{S} \leftarrow \chi(\mathbb{Z}_{q_1}^{n_3 \times m_1})$ and $\mathbf{E} \leftarrow \chi(\mathbb{Z}_{q_1}^{n_3 \times m_1})$. Then compute $\mathbf{C}^* = \mathbf{K} \cdot \mathbf{S} + \mathbf{E} \in \mathbb{Z}_{q_1}^{n_3 \times m_1}$. That is the exact generator formula of real matrix $\mathbf{C}$.
- Select a random user identity ID_r from the current info_{τ^*} . Next, use tpk to double-encrypt ID_r via the same operations in a real handshake, so as to produce ciphertexts $(\mathbf{c}_1^*, \mathbf{c}_2^*)$. By the semantic security of the underlying encryption, $(\mathbf{c}_1^*, \mathbf{c}_2^*)$ is computationally indistinguishable from the real ones.

⁶ Recall that *full anonymity* of AME ensures that the text leaks no information of user's identity. In other words, user's behavior is indistinguishable from a random simulation even if $\mathcal{A}$ has known the user's credential (see Section 4 for more details).

Table 2

Parameters and cost for our scheme.

Security level	δ	n	$\lceil \log q \rceil$	p	t	l_1	l_2	n_1	Handshake cost	Update cost	Tracing cost
80	1.0048	14	80	2^{20}	4	2240	2600	3050	7.29 MB	1.54 KB	10.93 MB
100	1.0042	15	90	2^{25}	4	3000	3400	3900	10.11 MB	1.86 KB	15.74 MB
128	1.0035	16	107	2^{32}	4	4240	4850	5520	15.89 MB	2.36 KB	26.47 MB

- With public input $pp^* = (\text{par}, \text{gpk}^*, \mathbf{u}_{\tau^*}, \mathbf{c}_1^*, \mathbf{c}_2^*)$, invoke $\text{Send}_{\text{hs}}(pp^*, \cdot, F(\mathbf{C}))$ to generate a text Σ^* , where the default item refers to applying the aforementioned simulator of ZKAoK in [50] within Send_{hs} . By the randomness of $F(\mathbf{C})$ and zero-knowledge of the used ZKAoK, Σ^* is computationally indistinguishable from the real one.
 - Sample $\mathbf{v}^* \xleftarrow{\$} \mathbb{Z}_q$, which is computationally indistinguishable from the real one.
- The output $(\text{Proof}^*, \mathbf{v}^*)$ is accordingly computationally indistinguishable from the real ones. $\square$

5.4. Performance analysis

Now we focus on the efficiency of our scheme, the concrete evaluation is shown as follows.

- group public key gpk has bit-size $(n + n_1 + n_2 + 4n\ell)k$ and credential cred has bit-size $nk + \ell \cdot (nk + 1)$.
- At each update epoch, group members download $(\ell + 1) \cdot (nk) + \ell$ bits.
- From system Π_{hs} , we know the total length of witness $\mathbf{x}_3$ is $n = 5\ell + 4n\ell + 2nk\ell + 4nk + \lceil \log(nk) \rceil + 2(n_1 + n_2)$, and the size of $\mathcal{L}_3$ is $l = 5\ell + 2nk\ell + 2n\ell + 4nk + \lceil \log(nk) \rceil + 2(n_1 + n_2)$. Thus, the text Σ_{hs} from Send_{hs} contains $(\log(2p + 1) + \kappa + (3l_1 + 2l_2 + 2n + 2l) \cdot k) \cdot t + (l_1 + n) \cdot k$ bits.
- Each ciphertext has two vectors in $\mathbb{Z}_q$, one of which is of length n_1 and the other is of length ℓ . Hence the two ciphertexts contain $2(n_1 + \ell) \cdot k$ bits; The authentication code $\mathbf{v}$ is a hash value of length k .
- Tracing result consists of an identity ID having ℓ bits, a proof π_{trace} with bit-size $(\log(2p + 1) + \kappa + (3l_1 + 2l_2 + 2n' + 2l') \cdot k) \cdot t + (l_1 + n') \cdot k$, where $n' = l' = (n_1 + n_2 + \lceil \log(n_1 + n_2 + 2) \rceil)\ell$, and a ciphertext $\mathbf{c}$ of size $(n_1 + \ell) \cdot k$.

Setting the parameters. We estimate the concrete cost of our FDSH via choosing suitable parameters. Since the underlying AME is constructed from the framework recalled in Section 2.4, from Theorem 3-6 we know that the security of our scheme relies on 6 assumptions:

$$\{\text{SIS}_{l_1, q, \beta_1}, \text{SIS}_{l_1, q, \beta_2}, \text{LWE}_{l_2, q, \sigma_1}, \text{SIS}_{n, q, \sqrt{m}}, \text{BLWE}_{n_1, q}, \text{BLWE}_{n_2, q}\},$$

where $\beta_1 = 16p\sqrt{l_1 + l_2 + n} \cdot (\sigma_2 + p\sigma_1)$, $\beta_2 = 16p\sqrt{l_1 + l_2 + l} \cdot (\sigma_2 + p\sigma_1)$.

Following work in [2, 15, 50], we examine the root Hermite factor (RHF) δ to estimate the hardness of involved assumptions. Set RHF as 1.0048 / 1.0042 / 1.0035 to achieve an 80 / 100 / 128-bit security, respectively, fix $\kappa = 256$, $N = 2^{10}$ and $n_1 = n_2$, then the corresponding parameters and cost are set as in Table 2 (recall $m = 2n \log q$). From this table, we know that our FDSH provides significant efficiency increase during an interactive handshake protocol (on the order of small megabytes), which makes it more practical when embedded in a synthetical network system.

6. Conclusion

In this work, we first formalize the model of FDSH with suitable security notions. Then, to achieve anonymous mutual authentication in a universal manner, a newly-defined primitive called anonymous message exchange and two generic approaches from ZKAoK are introduced, which may also be of independent cryptographic interest. Finally, combining a well-designed AME and other appropriate techniques, we propose the first lattice-based FDSH with another desired property known as deniability. The scheme greatly outweighs the existing post-quantum secure ones [3, 51] in terms of efficiency, security and functionality. One possible future work is to exploit more practical frameworks for FDSH.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Fangguo ZHANG reports financial support was provided by Guangdong Major Project of Basic and Applied Basic Research (2019B030302008). Fangguo ZHANG reports financial support was provided by the National Natural Science Foundation of China (No. 61972429). Yamin WEN reports financial support was provided by Guangdong Basic and Applied Basic Research Foundation (No. 2019A1515011797). Yamin WEN reports financial support was provided by the Opening Project of Guangdong Provincial Key Laboratory of Information Security Technology (2020B1212060078-09).

Acknowledgements

This work is supported by Guangdong Major Project of Basic and Applied Basic Research (2019B030302008) and the National Natural Science Foundation of China (No. 61972429) and Guangdong Basic and Applied Basic Research Foundation (No. 2019A1515011797) and the Opening Project of Guangdong Provincial Key Laboratory of Information Security Technology (2020B1212060078-09).

References

- [1] M. Ajtai, Generating hard instances of lattice problems (extended abstract), in: G.L. Miller (Ed.), STOC 1996, ACM, 1996, pp. 99–108.
- [2] M.R. Albrecht, R. Player, S. Scott, On the concrete hardness of learning with errors, *J. Math. Cryptol.* 9 (3) (2015) 169–203.
- [3] Z. An, Z. Zhang, Y. Wen, F. Zhang, Lattice-based secret handshakes with reusable credentials, in: D. Gao, Q. Li, X. Guan, X. Liao (Eds.), ICICS 2021, Springer, 2021, pp. 231–248.
- [4] G. Ateniese, J. Kirsch, M. Blanton, Secret handshakes with dynamic and fuzzy matching, in: NDSS 2007, The Internet Society, 2007.
- [5] D. Balfanz, G. Durfee, N. Shankar, D.K. Smetters, J. Staddon, H. Wong, Secret handshakes from pairing-based key agreements, in: 2003 IEEE Symposium on Security and Privacy, IEEE Computer Society, 2003, pp. 180–196.
- [6] D. Boneh, H. Shacham, Group signatures with verifier-local revocation, in: V. Atluri, B. Pfizmann, P.D. McDaniel (Eds.), CCS 2004, ACM, 2004, pp. 168–177.
- [7] J. Bootle, A. Cerulli, P. Chaidos, E. Ghadafi, J. Groth, Foundations of fully dynamic group signatures, in: ACNS 2016, Springer, 2016, pp. 117–136.
- [8] J.W. Bos, C. Costello, L. Ducas, I. Mironov, M. Naehrig, V. Nikolaenko, A. Raghunathan, D. Stebila, Frodo: take off the ring! Practical, quantum-secure key exchange from LWE, in: CCS 2016, ACM, 2016, pp. 1006–1018.
- [9] Z. Brakerski, A. Langlois, C. Peikert, O. Regev, D. Stehlé, Classical hardness of learning with errors, in: D. Boneh, T. Roughgarden, J. Feigenbaum (Eds.), STOC 2013, ACM, 2013, pp. 575–584.
- [10] G. Brassard, D. Chaum, C. Crépeau, Minimum disclosure proofs of knowledge, *J. Comput. Syst. Sci.* 37 (2) (1988) 156–189.
- [11] E.F. Brickell, D. Pointcheval, S. Vaudenay, M. Yung, Design validations for discrete logarithm based signature schemes, in: PKC 2000, Springer, 2000, pp. 276–292.
- [12] C. Castelluccia, S. Jarecki, G. Tsudik, Secret handshakes from ca-oblivious encryption, in: P.J. Lee (Ed.), ASIACRYPT 2004, Springer, 2004, pp. 293–307.
- [13] P. Cayrel, P. Véron, S.M.E.Y. Alaoui, A zero-knowledge identification scheme based on the q-ary syndrome decoding problem, in: SAC 2010, Springer, 2010, pp. 171–186.
- [14] M.F. Esgin, R. Steinfeld, A. Sakzad, J.K. Liu, D. Liu, Short lattice-based one-out-of-many proofs and applications to ring signatures, in: R.H. Deng, V. Gauthier-Umaña, M. Ochoa, M. Yung (Eds.), ACNS 2019, Springer, 2019, pp. 67–88.
- [15] N. Gama, P.Q. Nguyen, Predicting lattice reduction, in: N.P. Smart (Ed.), EUROCRYPT 2008, Springer, 2008, pp. 31–51.
- [16] C. Gentry, C. Peikert, V. Vaikuntanathan, Trapdoors for hard lattices and new cryptographic constructions, in: STOC 2008, ACM, 2008, pp. 197–206.
- [17] S. Goldwasser, S. Micali, C. Rackoff, The knowledge complexity of interactive proof-systems (extended abstract), in: STOC 1985, ACM, 1985, pp. 291–304.
- [18] D. He, N. Kumar, H. Wang, L. Wang, K.R. Choo, A.V. Vinel, A provably-secure cross-domain handshake scheme with symptoms-matching for mobile healthcare social network, *IEEE Trans. Dependable Secure Comput.* 15 (4) (2018) 633–645.
- [19] L. Hou, J. Lai, L. Liu, Secret handshakes with dynamic expressive matching policy, in: J.K. Liu, R. Steinfeld (Eds.), ACISP 2016, Springer, 2016, pp. 461–476.
- [20] S. Jarecki, J. Kim, G. Tsudik, Authentication for paranoids: multi-party secret handshakes, in: J. Zhou, M. Yung, F. Bao (Eds.), ACNS 2006, 2006, pp. 325–339.
- [21] S. Jarecki, J. Kim, G. Tsudik, Group secret handshakes or affiliation-hiding authenticated group key agreement, in: M. Abe (Ed.), CT-RSA 2007, Springer, 2007, pp. 287–308.
- [22] S. Jarecki, X. Liu, Unlinkable secret handshakes and key-private group key management schemes, in: J. Katz, M. Yung (Eds.), ACNS 2007, Springer, 2007, pp. 270–287.
- [23] S. Jarecki, X. Liu, Private mutual authentication and conditional oblivious transfer, in: CRYPTO 2009, Springer, 2009, pp. 90–107.
- [24] Y. Kawai, K. Yoneyama, K. Ohta, Secret handshake: strong anonymity definition and construction, in: F. Bao, H. Li, G. Wang (Eds.), ISPEC 2009, Springer, 2009, pp. 219–229.
- [25] F. Laguillaumie, A. Langlois, B. Libert, D. Stehlé, Lattice-based group signatures with logarithmic signature size, in: K. Sako, P. Sarkar (Eds.), ASIACRYPT 2013, Springer, 2013, pp. 41–61.
- [26] A. Langlois, S. Ling, K. Nguyen, H. Wang, Lattice-based group signature scheme with verifier-local revocation, in: PKC 2014, Springer, 2014, pp. 345–361.
- [27] B. Libert, S. Ling, F. Mouhartem, K. Nguyen, H. Wang, Signature schemes with efficient protocols and dynamic group signatures from lattice assumptions, in: J.H. Cheon, T. Takagi (Eds.), ASIACRYPT 2016, 2016, pp. 373–403.
- [28] B. Libert, S. Ling, K. Nguyen, H. Wang, Zero-knowledge arguments for lattice-based accumulators: logarithmic-size ring signatures and group signatures without trapdoors, in: M. Fischlin, J. Coron (Eds.), EUROCRYPT 2016, Springer, 2016, pp. 1–31.
- [29] R. Lindner, C. Peikert, Better key sizes (and attacks) for lwe-based encryption, in: A. Kiayias (Ed.), CT-RSA 2011, Springer, 2011, pp. 319–339.
- [30] S. Ling, K. Nguyen, D. Stehlé, H. Wang, Improved zero-knowledge proofs of knowledge for the ISIS problem, and applications, in: K. Kurosawa, G. Hanaoka (Eds.), PKC 2013, Springer, 2013, pp. 107–124.
- [31] S. Ling, K. Nguyen, H. Wang, Y. Xu, Lattice-based group signatures: achieving full dynamicity with ease, in: D. Gollmann, A. Miyaji, H. Kikuchi (Eds.), ACNS 2017, Springer, 2017, pp. 293–312.
- [32] V. Lyubashevsky, Lattice-based identification schemes secure under active attacks, in: R. Cramer (Ed.), PKC 2008, Springer, 2008, pp. 162–179.
- [33] V. Lyubashevsky, Fiat-Shamir with aborts: applications to lattice and factoring-based signatures, in: M. Matsui (Ed.), ASIACRYPT 2009, Springer, 2009, pp. 598–616.
- [34] V. Lyubashevsky, Lattice signatures without trapdoors, in: D. Pointcheval, T. Johansson (Eds.), EUROCRYPT 2012, Springer, 2012, pp. 738–755.
- [35] D. Micciancio, C. Peikert, Hardness of SIS and LWE with small parameters, in: R. Canetti, J.A. Garay (Eds.), CRYPTO 2013, Springer, 2013, pp. 21–39.
- [36] Y. Michalevsky, S. Nath, J. Liu, Mashable: mobile applications of secret handshakes over bluetooth LE, in: Y. Chen, M. Gruteser, Y.C. Hu, K. Sundaresan (Eds.), MobiCom 2016, ACM, 2016, pp. 387–400.
- [37] T. Nakanishi, N. Funabiki, Verifier-local revocation group signature schemes with backward unlinkability from bilinear maps, in: B.K. Roy (Ed.), ASIACRYPT 2005, Springer, 2005, pp. 533–548.
- [38] M. Naor, M. Yung, Public-key cryptosystems provably secure against chosen ciphertext attacks, in: H. Ortiz (Ed.), STOC 1990, ACM, 1990, pp. 427–437.
- [39] S. Panja, S. Dutta, K. Sakurai, Deniable secret handshake protocol - revisited, in: L. Barolli, M. Takizawa, F. Xhafa, T. Enokido (Eds.), AINA 2019, Springer, 2019, pp. 1266–1278.
- [40] M.D. Raimondo, R. Gennaro, New approaches for deniable authentication, in: CCS 2005, ACM, 2005, pp. 112–121.
- [41] M.D. Raimondo, R. Gennaro, H. Krawczyk, Deniable authentication and key exchange, in: CCS 2006, ACM, 2006, pp. 400–409.

- [42] O. Regev, On lattices, learning with errors, random linear codes, and cryptography, in: H.N. Gabow, R. Fagin (Eds.), STOC 2005, ACM, 2005, pp. 84–93.
- [43] J. Stern, A new paradigm for public key identification, *IEEE Trans. Inf. Theory* 42 (6) (1996) 1757–1768.
- [44] Y. Tian, Y. Li, Y. Zhang, N. Li, G. Yang, Y. Yu, DSH: deniable secret handshake framework, in: ISPEC 2018, Springer, 2018, pp. 341–353.
- [45] G. Tsudik, S. Xu, A flexible framework for secret handshakes, in: G. Danezis, P. Golle (Eds.), PET 2006, Springer, 2006, pp. 295–315.
- [46] N. Unger, I. Goldberg, Deniable key exchanges for secure messaging, in: I. Ray, N. Li, C. Kruegel (Eds.), CCS 2015, ACM, 2015, pp. 1211–1223.
- [47] Y. Wen, F. Zhang, A new revocable secret handshake scheme with backward unlinkability, in: J. Camenisch, C. Lambrinoudakis (Eds.), EuroPKI 2010, Springer, 2010, pp. 17–30.
- [48] Y. Wen, F. Zhang, H. Wang, Z. Gong, Y. Miao, Y. Deng, A new secret handshake scheme with multi-symptom intersection for mobile healthcare social networks, *Inf. Sci.* 520 (2020) 142–154.
- [49] S. Xu, M. Yung, k-anonymous secret handshakes with reusable credentials, in: CCS 2004, ACM, 2004, pp. 158–167.
- [50] R. Yang, M.H. Au, Z. Zhang, Q. Xu, Z. Yu, W. Whyte, Efficient lattice-based zero-knowledge arguments with standard soundness: construction and applications, in: A. Boldyreva, D. Micciancio (Eds.), CRYPTO 2019, Springer, 2019, pp. 147–175.
- [51] Z. Zhang, F. Zhang, H. Tian, CSH: a post-quantum secret handshake scheme from coding theory, in: L. Chen, N. Li, K. Liang, S.A. Schneider (Eds.), ESORICS 2020, Springer, 2020, pp. 317–335.
- [52] L. Zhou, W. Susilo, Y. Mu, Three-round secret handshakes based on elgamal and DSA, in: K. Chen, R.H. Deng, X. Lai, J. Zhou (Eds.), ISPEC 2006, Springer, 2006, pp. 332–342.
- [53] L. Zhou, W. Susilo, Y. Mu, New construction of group secret handshakes based on pairings, in: S. Qing, H. Imai, G. Wang (Eds.), ICICS 2007, Springer, 2007, pp. 16–30.